分布式系统

复习串讲

课程内容

讲次	课程内容	讲次	课程内容
第1讲	概述	第11讲	容错机制
第2讲	体系架构	第12讲	共识协议
第3讲	进程模型	第13讲	共识协议
第4讲	通信机制	第14讲	云计算与虚拟化
第5讲	命名机制	第15讲	Hadoop系统（大数据处理）
第6讲	协同与同步	第16讲	Hadoop系统（大数据处理）
第7讲	协同与同步	第17讲	区块链
第8讲	一致性与复制	第18讲	期末复习
第9讲	一致性与复制		
第10讲	容错机制		

Part I 基本概念

- 分布式系统定义
- 分布式系统目标
- 系统分类
- 系统架构
- 通信中间件
- 虚拟化
- 命名
- Causality
- 容错、失效模型

分布式系统定义

- **一个简单的定义**

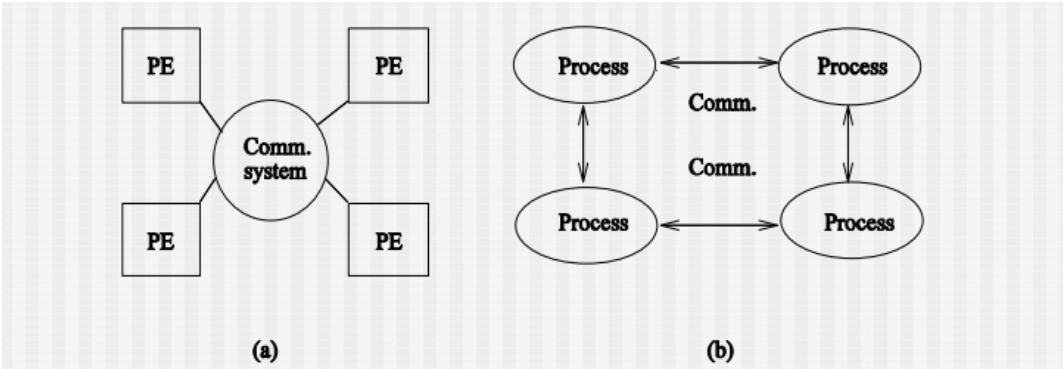
  分布式系统是若干**独立自主计算机**的集合，这些计算机对于用户来说像是**单个耦合**系统。



- **Leslie Lamport的定义**

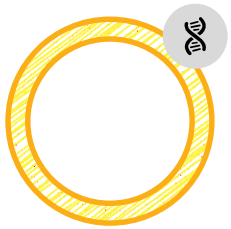
  “A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.”

物理分布，逻辑集中
个体独立，整体统一

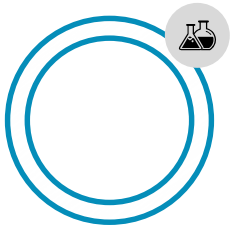


System structure from the physical (a) or logical point of view (b)

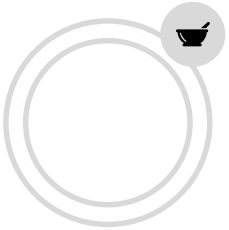
分布式系统的目标



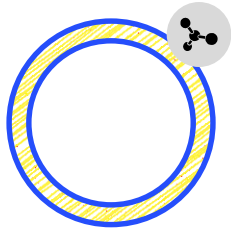
使资源可访问
让用户方便地访问资源



透明性
隐藏资源在网络上的分布



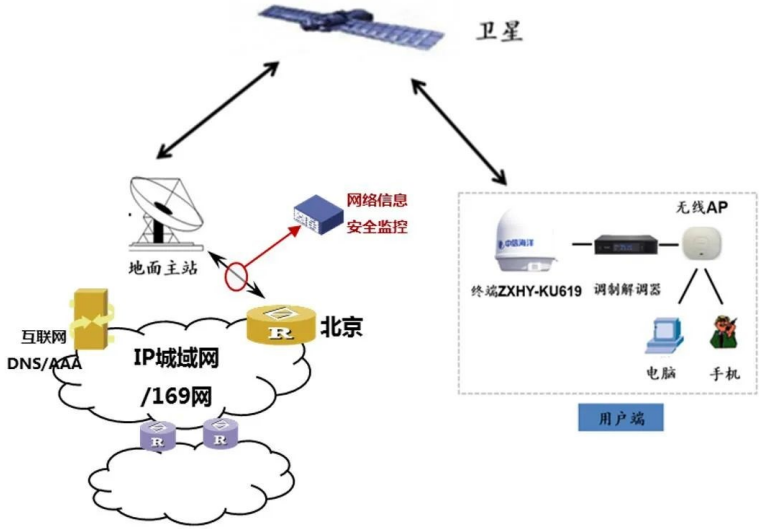
开放性
访问接口的标准化



可扩展性
规模、地域、管理上可扩展

资源访问Resource Access

- 文件、数据、算力…
- 普适、随时访问







透明性Transparency

- 隐藏分布式系统的“分布性”

透明性	说明
访问	隐藏数据表示形式的不同以及资源访问方式的不同
位置	隐藏资源所在位置
迁移	异常资源是否移动到另一个位置
重定位	隐藏资源是否在使用过程中移动到另一个位置
复制	隐藏是否对资源进行复制
并发	隐藏资源是否由相互竞争的用户共享
故障	隐藏资源的故障和恢复
持久化	隐藏数据在主存和磁盘这一事实

- 透明性会影响系统性能
  - 完全透明难以实现的，而且并不可取
- 完全透明难实现：
  - 完全隐藏网络和节点的失效是不可能的
  - 容错和一致性机制需要时间
- 不透明性有时有益：
  - 及时发现系统的性能问题
  - 有时候需要暴露系统特征

开放性Openness

- 定义
  - 系统根据一系列准则来提供服务，这些准则描述了所提供服务的语法和语义 （标准化）
- 具体要求
  - 良好定义的接口；
  - 良好的互操作性；
  - 支持可移植性；
  - 容易扩展（extensibility）

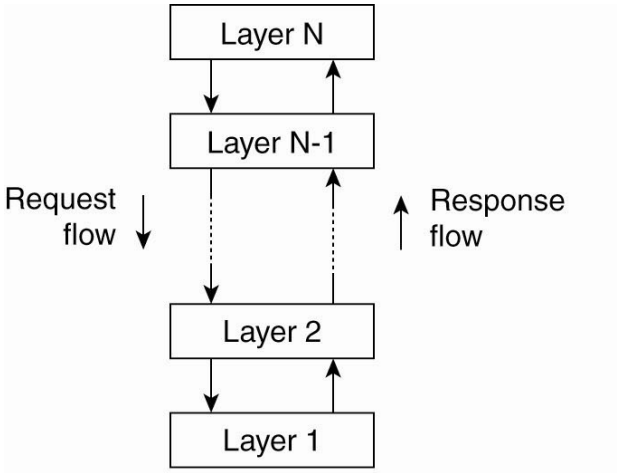
可扩展性Scalability

- 可以伸缩、增减的能力
- 三个维度的扩展性：
  - 规模Size: be able to easily add more users and resources to a system
  - 地理Geography: be able to handle users and resources that are far apart
  - 管理Administrative: be able to manage even if it spans independent administrative organizations
- 规模是最基本、最常用的扩展性
- 地理和管理扩展性日益重要

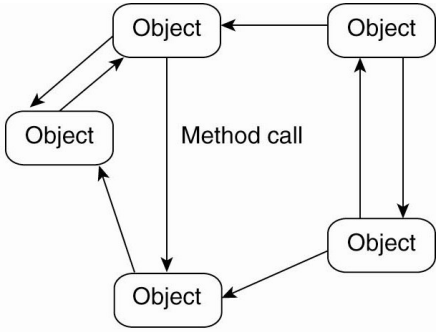
分布式系统分类

- Distributed (High Performance) Computing Systems
  - Cluster Computing Systems
  - Grid Computing Systems
  - Cloud Computing Systems
- Distributed Information Systems
  - Transaction Processing Systems
  - Enterprise Application Integration
- Distributed Pervasive Systems
  - Ubiquitous Computing Systems
  - Mobile Computing Systems
  - Wireless Sensor Networks

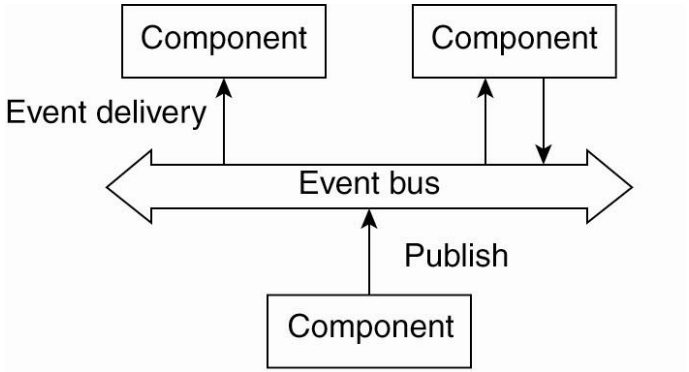
Architectural Styles （样式）



Layered architectures

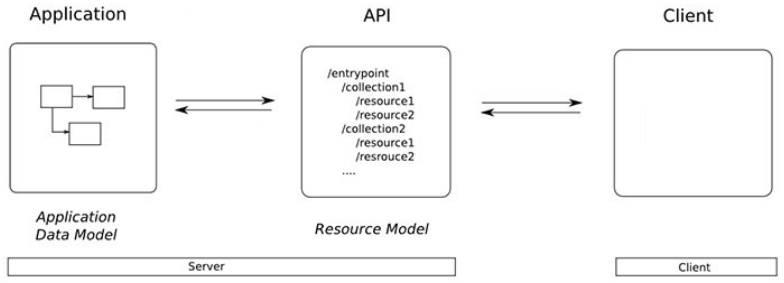


Object-based architectures

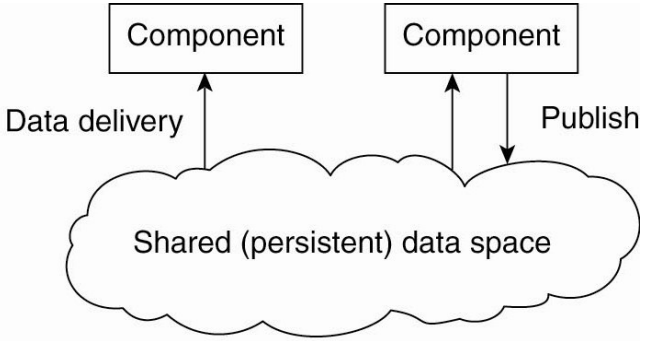


Event-based architectures

Pub-Sub



Resource-centered architectures



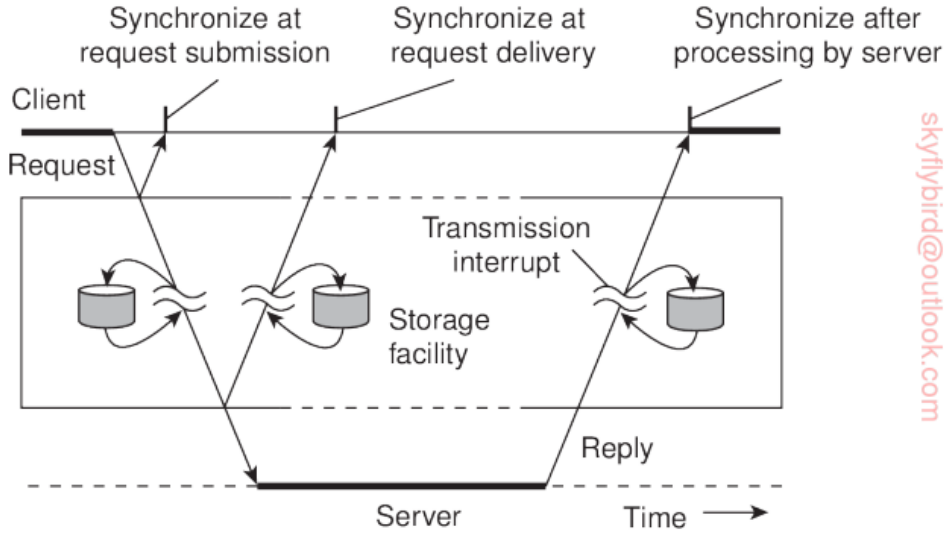
Data-shared architectures

不同类型体系结构

- Centralized architectures
  - Application layering
  - Multitiered architectures
- Decentralized architectures
  - Structured peer-to-peer
  - Unstructured peer-to-peer
  - Hierarchically organized peer-to-peer
- Hybrid architectures
  - Edge-server systems
  - Collaborative distributed systems

通信中间件

- 提供附加的、通用性的服务功能
  - 包含丰富的（应用层）通信协议；
  - 包装/解包装数据，对于系统集成非常重要；
  - 命名协议，允许资源的共享；
  - 扩展机制，例如复制和缓存。



skyflybird@outlook.com

Figure 4.4: Viewing middleware as an intermediate (distributed) service in application-level communication.

虚拟化技术

(a) 进程虚拟机 (Process VM)，如JVM

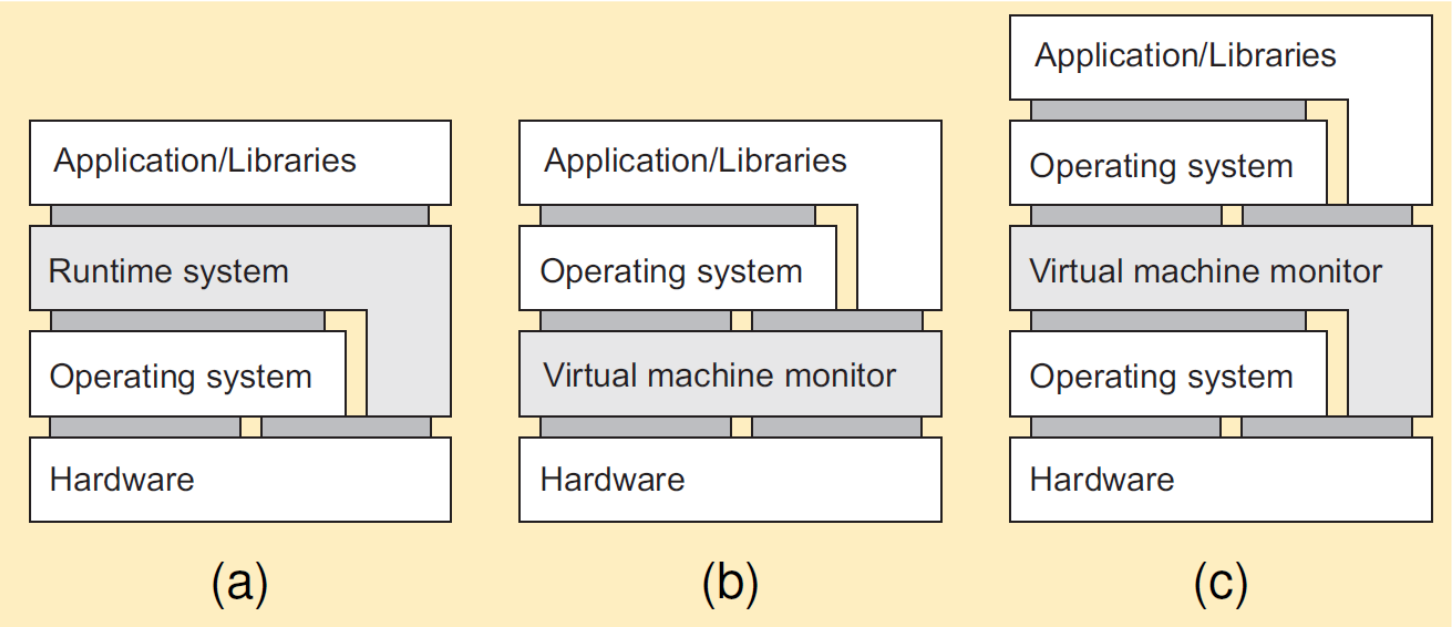
- 分离的指令集合，实际是运行在OS之上的解释器

(b) 原生虚拟机监控器 (Native VMM)

- 底层指令集，允许多样化的Guest OS

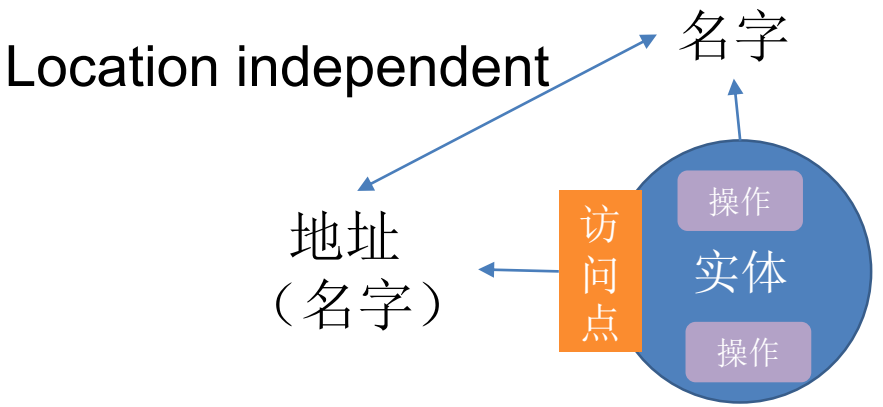
(c) 主机虚拟机监控器 (Hosted VMM)

- 利用Host OS能力简化VMM



名称/名字

- 名称Name：
  - 用名字标识分布式系统中的实体对象
  - 由字符组成的字符串
- 实体Entity：
  - 可以是分布式系统中的任何事物，抽象的或者实际的
  - 如：主机、用户、消息等。
- 访问点Access point：
  - 访问实体的的入口或途径
  - 是一种特殊实体
- 地址Address：
  - 用于访问实体的地址
  - 其实是实体的访问点的名称



Happens-before关系

- Happens-before 关系 a->b
- 也称为因果关系（Causality）
  - P1：如果 a 和 b是同一个进程中的两个事件，并且 a 在 b之前到达，则有：a->b
  - P2：如果 a 是消息的发送者，b是消息的接收者，则 a->b
  - 传递性：如果 a->b 并且 b->c 则 a->c
- 本质
  - 所有的进程并不一定在时间上达成一致，而只需要在时间发生顺序上达成一致，也就是需要“排序”
- 并发（concurrent）关系
  - a->b和b->a均不成立

容错概念

- 容错(Fault Tolerance)
  - 容错，容许系统的某些组件失效，而不会“严重”影响整体性能。故障发生时，系统可以进行恢复，同时可以继续进行任务处理操作。能够容错的系统，是“可依赖的”、“可靠的”（Dependable）。
- Dependability

Requirement	Description
Availability	Readiness for usage
Reliability	Continuity of service delivery
Safety	Very low probability of catastrophes
Maintainability	How easy can a failed system be repaired

失效模型（故障模式）

Type	Description of server's behavior
Crash failure	Halts, but is working correctly until it halts
Omission failure	Fails to respond to incoming requests
Receive omission	Fails to receive incoming messages
Send omission	Fails to send messages
Timing failure	Response lies outside a specified time interval
Response failure	Response is incorrect
Value failure	The value of the response is wrong
State-transition failure	Deviates from the correct flow of control
Arbitrary failure	May produce arbitrary responses at arbitrary times

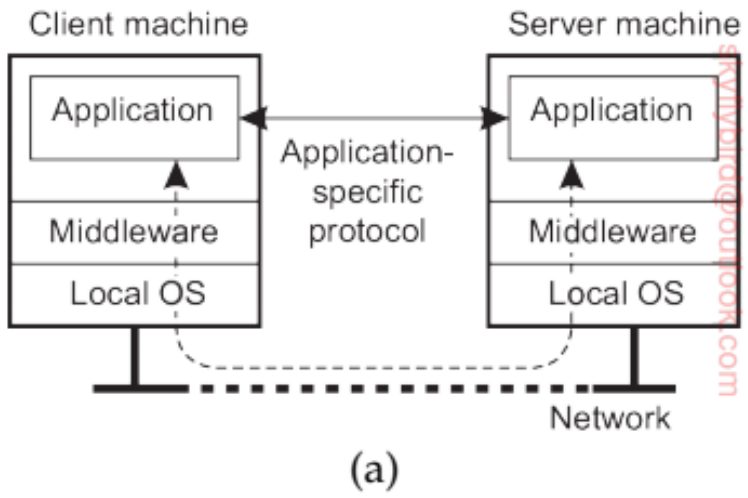
Arbitrary Failure （Byzantine Failure）：可以是恶意的，也可以是非恶意的，可能有各种表现。

Part II 基本技术

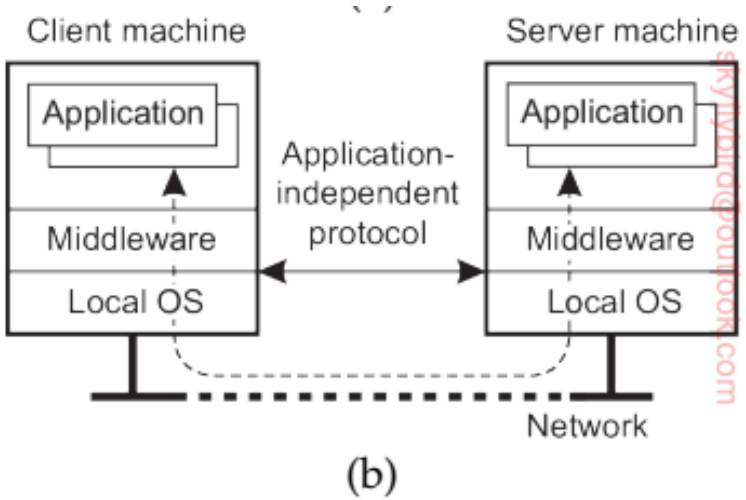
- 客户端与服务器
- 进程间通信
- 命名系统
- 协同与同步
- 副本与一致性
- 容错技术

客户端

- 客户端（Client）
  - to provide the means for users to interact with remote servers
- 网络化用户接口
  - App specific vs. App independent



(a)



(b)

服务器

- Iterative server迭代服务器:
  - One single server process provides the service for client.
  - Requests from multiple clients handled one by one sequentially.
- Concurrent server并发服务器:
  - Server process creates a child process/thread to handle each request.
  - It then returns to wait for a new request.
  - Multiple requests are handled concurrently.
- Stateful server: load is on server.
  - Server saves information about status of clients.
- Stateless server: load is on client. (RESTful)
  - Server maintains no information about status of clients.

Server Discovery

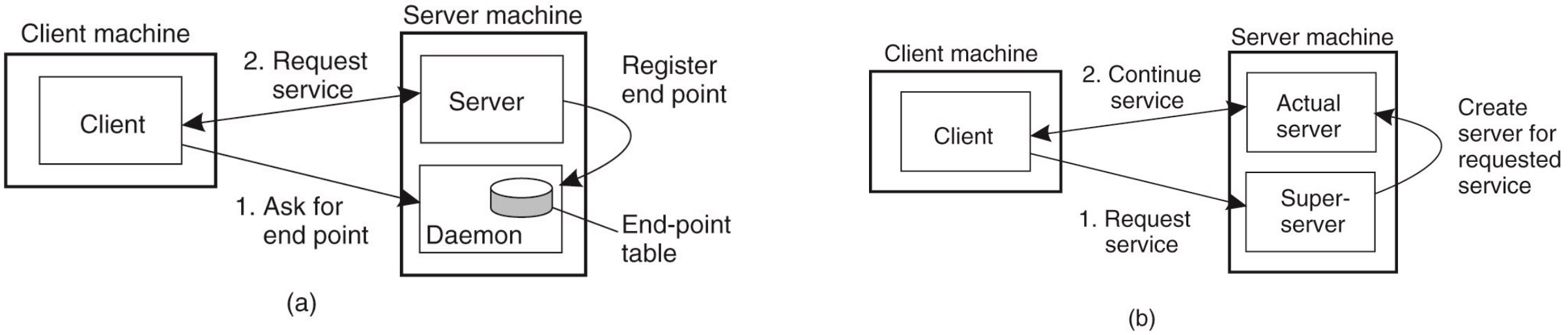
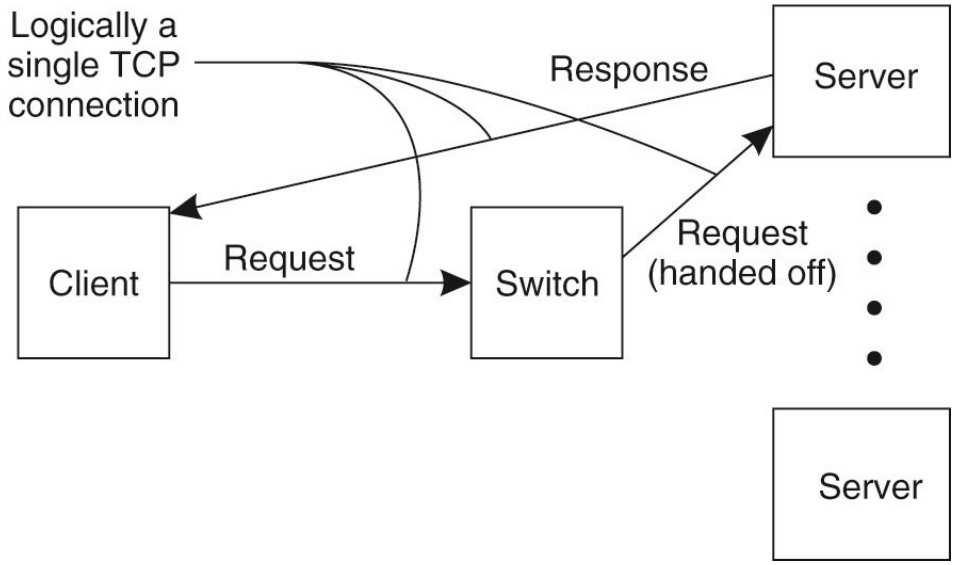


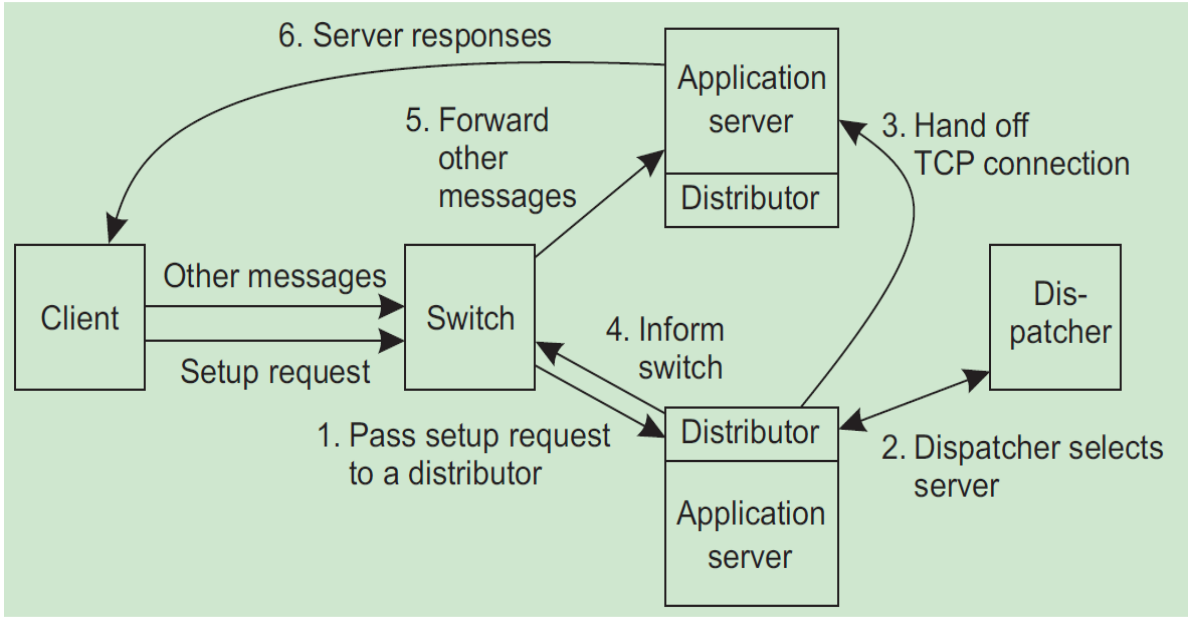
Figure 3-11. (a) Client-to-server binding using a daemon.
(b) Client-to-server binding using a superserver.

Server Clusters

Figure 3-13. The principle of TCP handoff (requires IP forwarding and IP rewriting).

传输层+请求内容混合分发

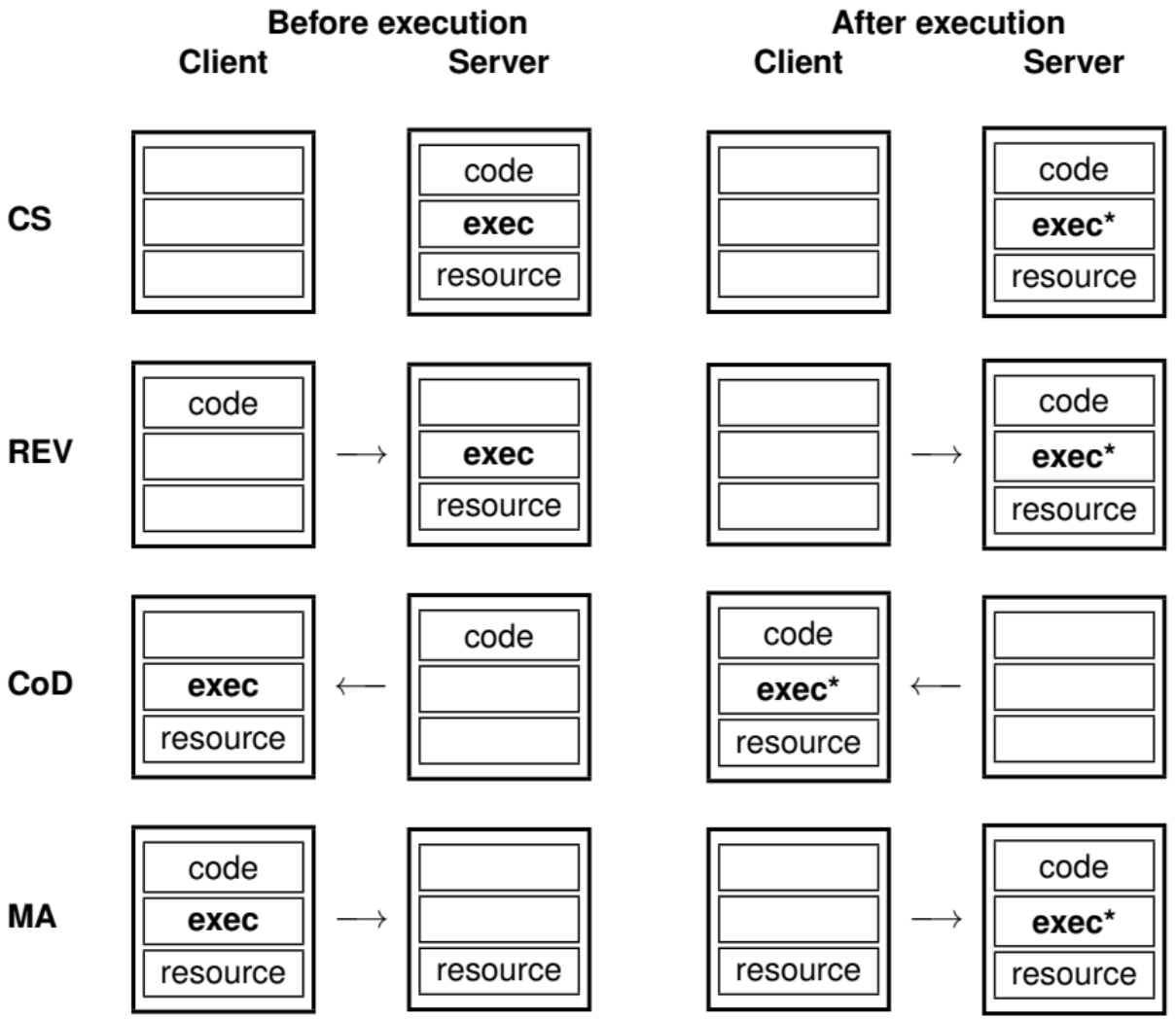




Code Migration Paradigms

- 四种样式

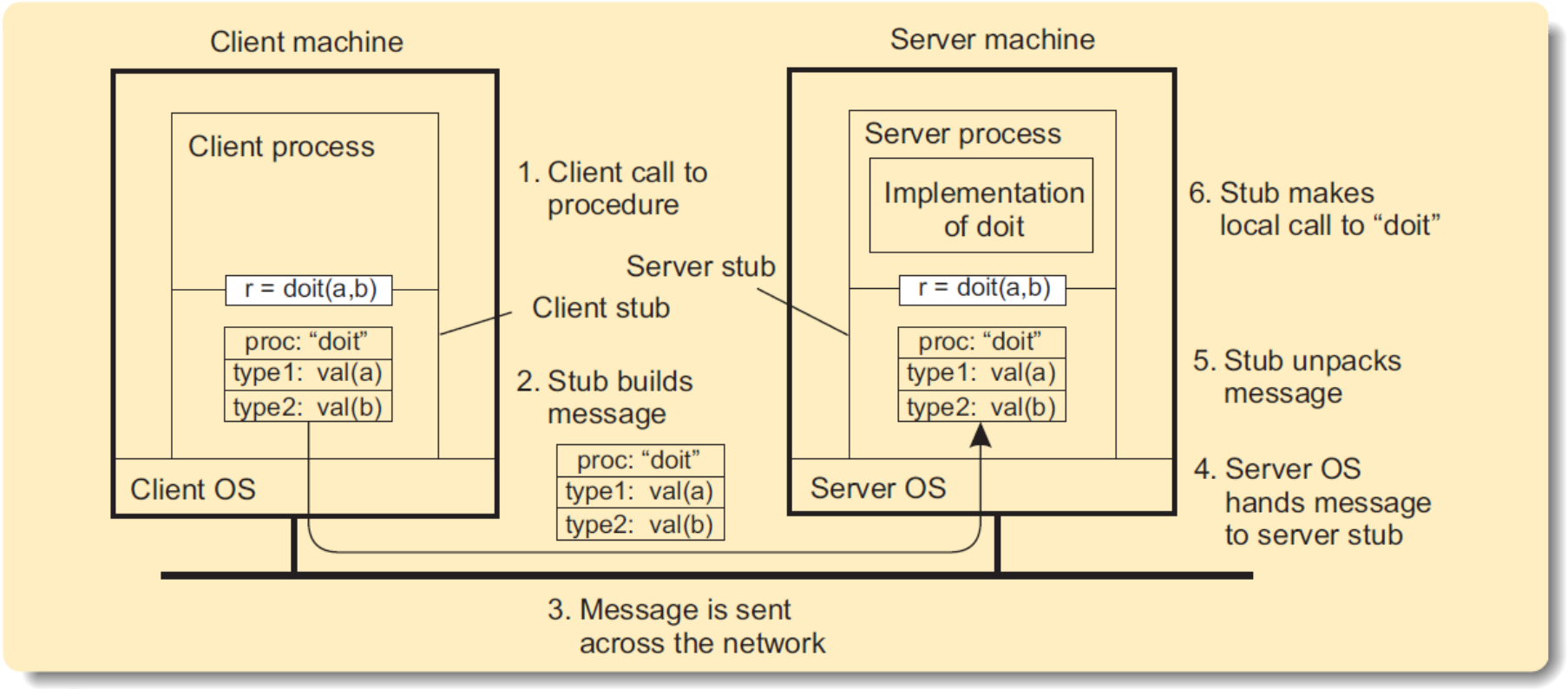
CS: Client-Server
CoD: Code-on-demand
REV: Remote evaluation
MA: Mobile agents



IPC Methods

- RPC:
  - Transparent
  - Synchronous nature
- Messaging
  - Socket: transient
    - General purpose
    - Simple, no powerful interfaces
  - MPI: transient
    - Parallel computing, mainly
    - Powerful with proprietary communication libraries, not robust
  - Message queuing: persistent
    - Asynchronous nature
- Streaming
  - For continuous data
- Multicasting
  - Overlay-base (established paths)
  - Gossip-based: probabilistic

Remote Procedure Calls



1. Client procedure calls client stub.
2. Stub builds message; calls local OS.
3. OS sends message to remote OS.
4. Remote OS gives message to stub.
5. Stub unpacks parameters; calls server.
6. Server does local call; returns result to stub.
7. Stub builds message; calls OS.
8. OS sends message to client's OS.
9. Client's OS gives message to stub.
10. Client stub unpacks result; returns to client.

Client-Server Binding

- One approach：directory server
  - First：Locate the server’s machine.
  - Second：Locate the server (i.e., the process) on that machine.

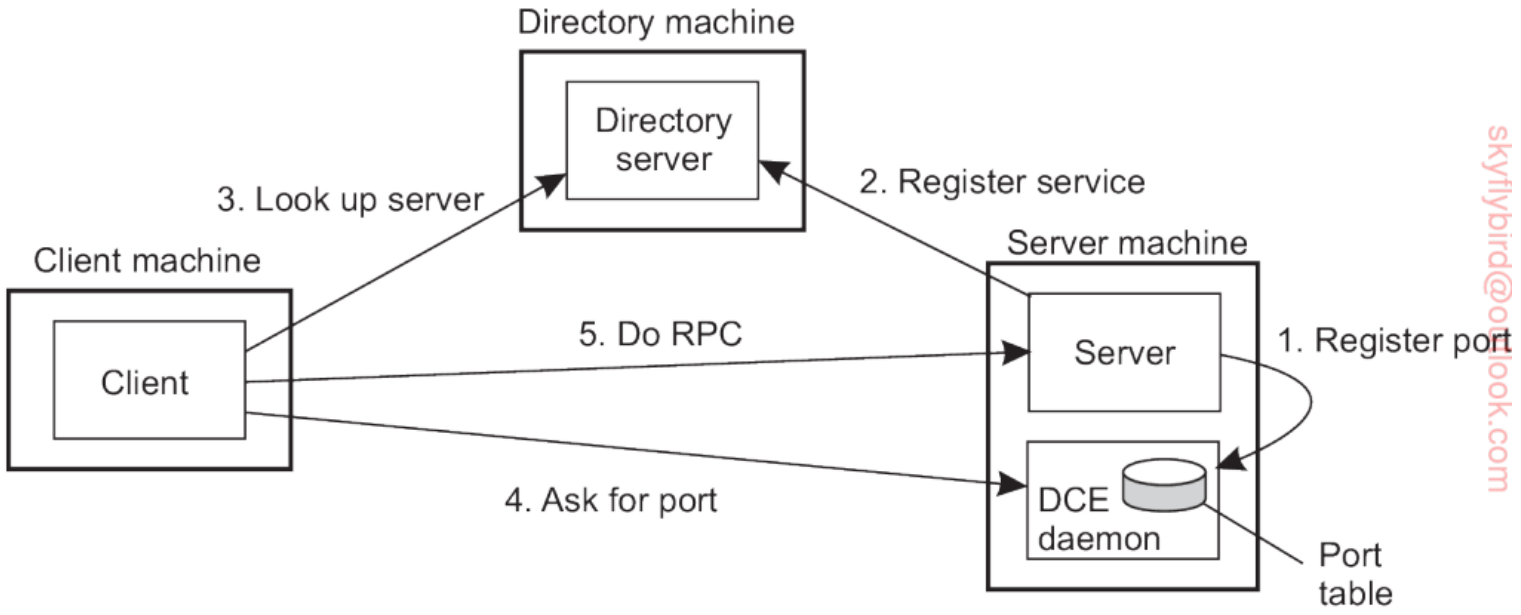
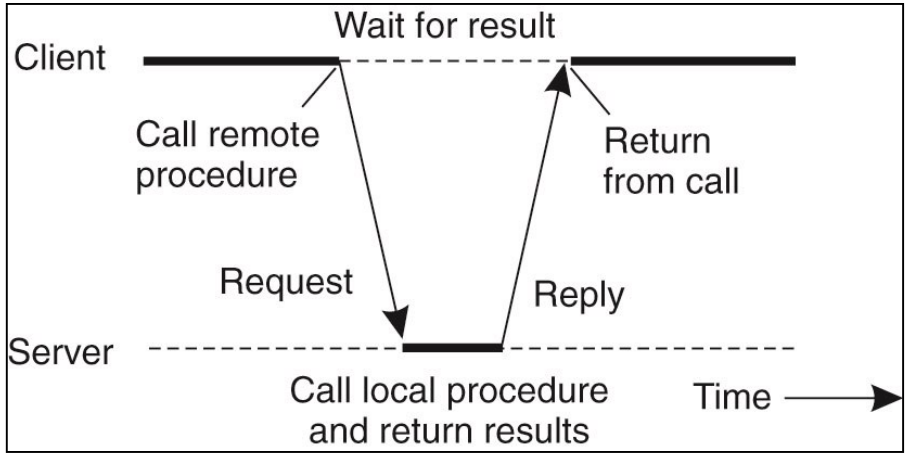


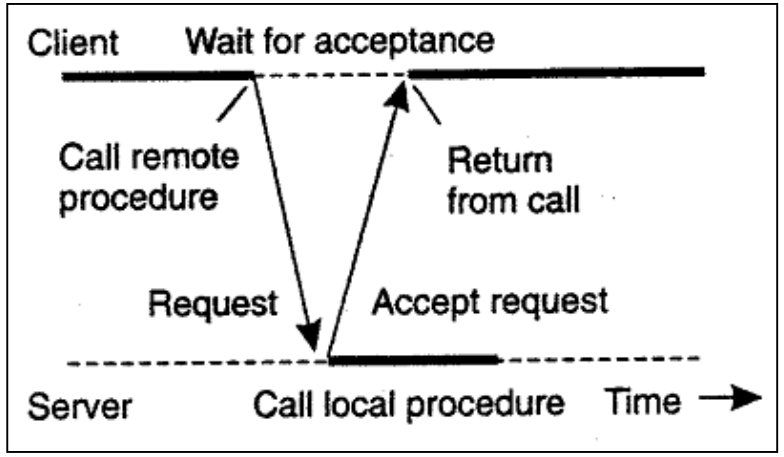
Figure 4.17: Client-to-server binding in DCE.

skyflybird@outlook.com

Asynchronous RPC

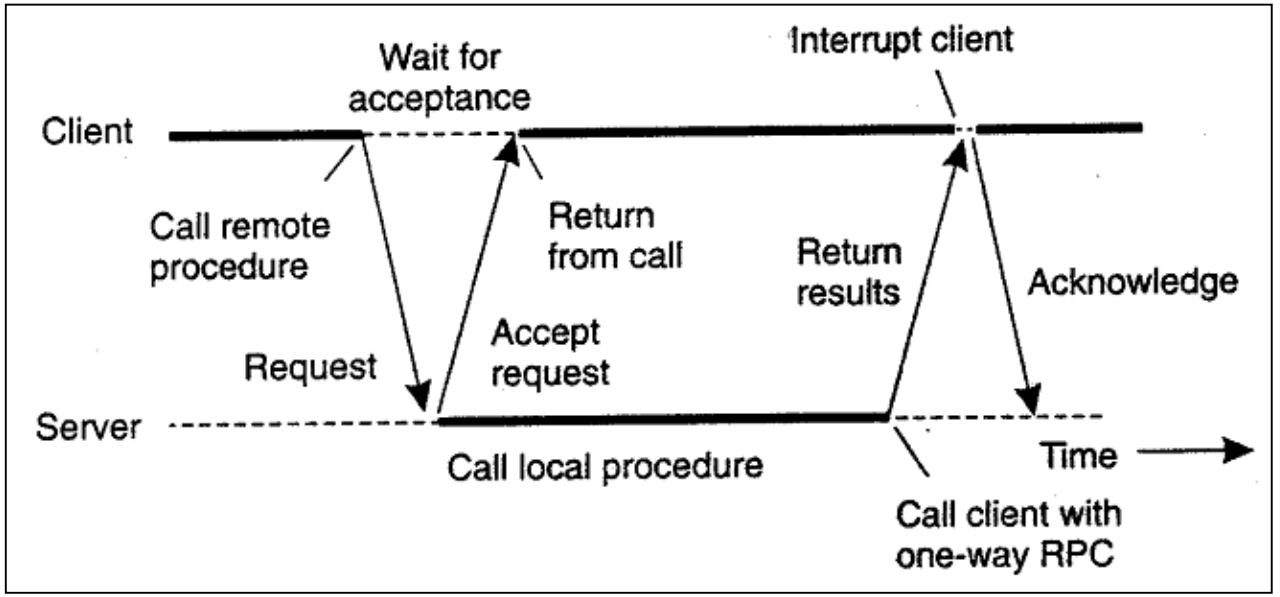


Common RPC



Asynchronous RPC

Deferred asynchronous RPC



面向消息的通信

- Transient
  - Socket-based message passing
  - ZeroMQ: Implementing messaging patterns
  - Message-Passing Interface (MPI)
- Persistent
  - Message queuing

MQ一般架构：Queue level addressing

- Queue manager + local queue+ application
- 如何发送和交付消息？三个技术问题：
  - 目的队列的地址？逻辑地址与位置无关
    - Name-address (host+port)，放在一个表/库中
  - QM如何得到name-address映射信息？
    - 配置然后复制，简单但是低效、维护开销大（不一致性）

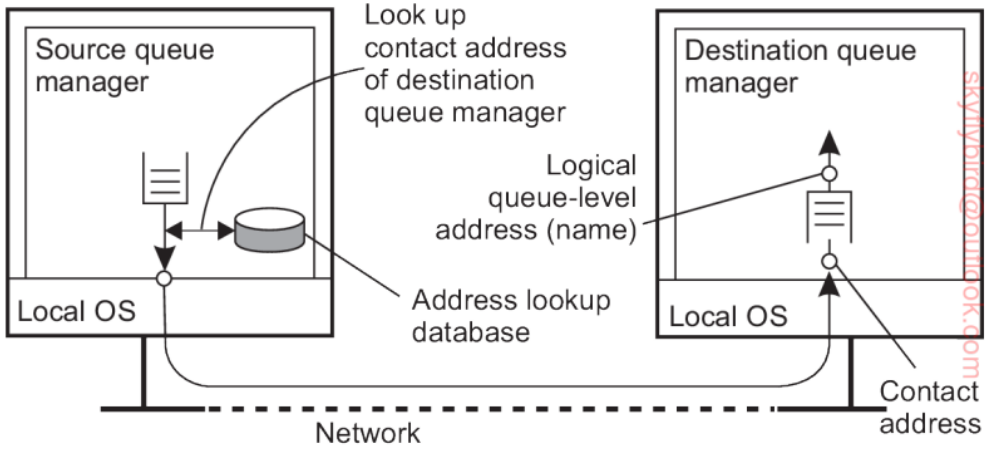
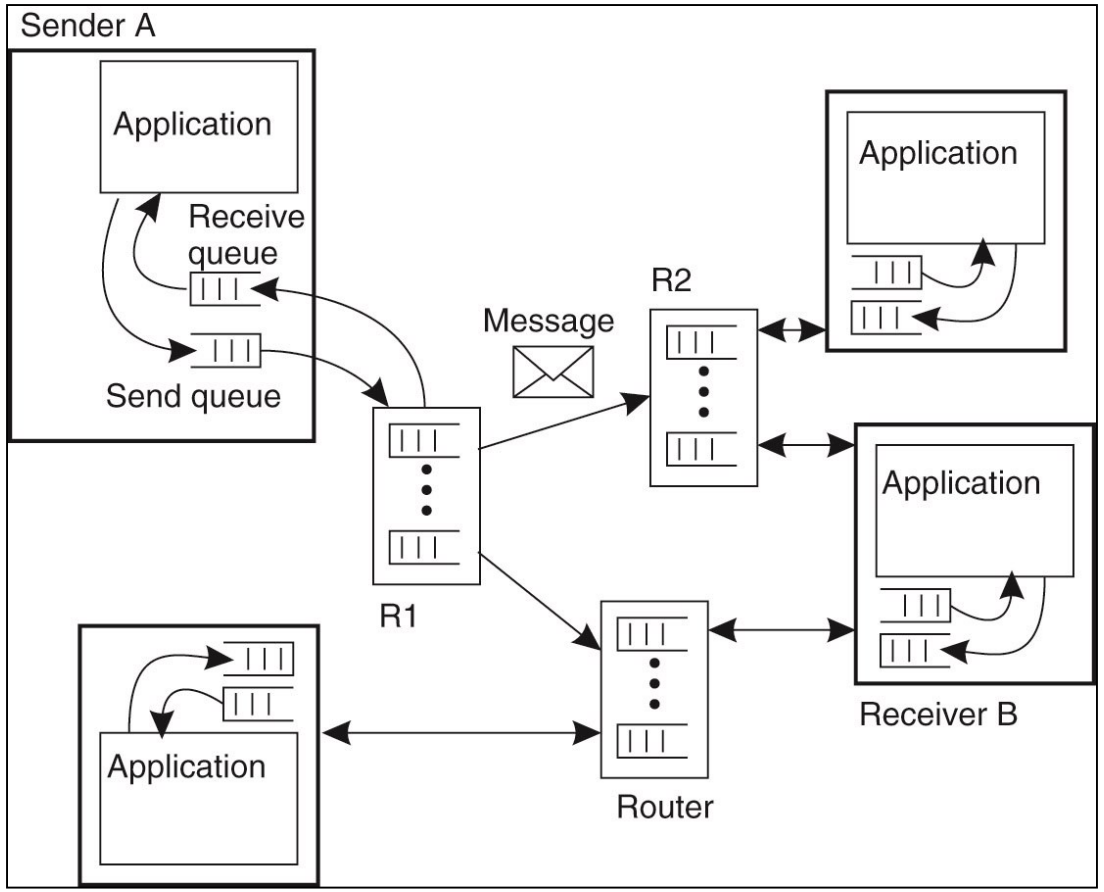


Figure 4.28: The relationship between queue-level naming and network-level addressing.

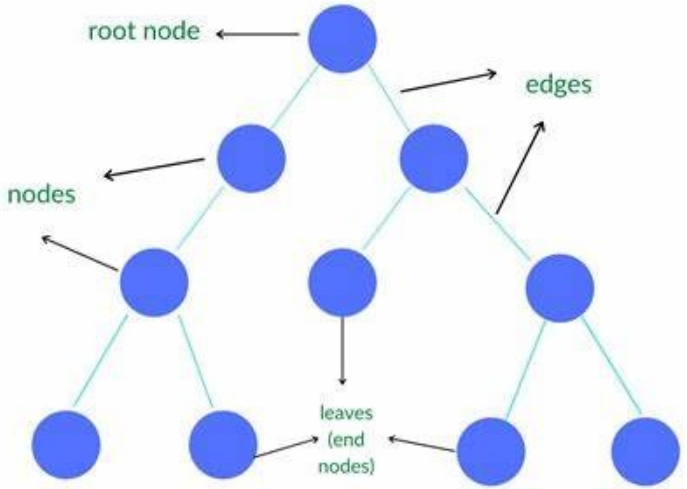
MQ消息路由架构

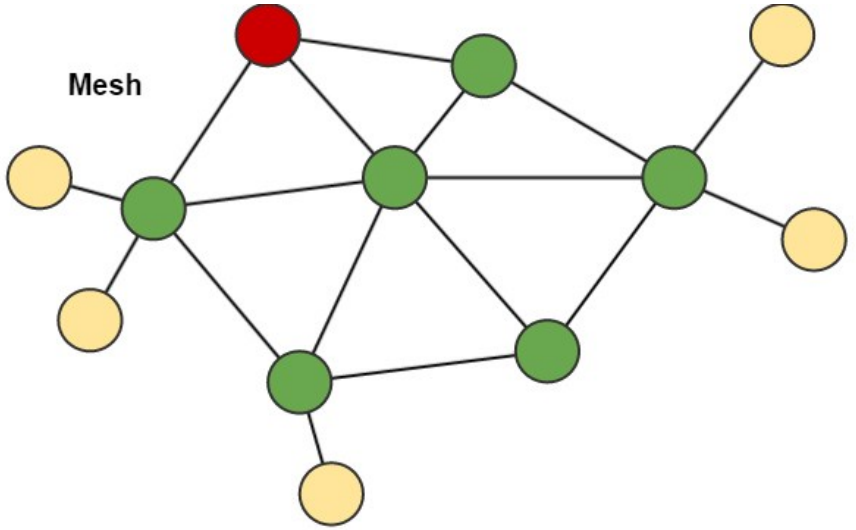
- 消息路由器进行消息路由、转发
- 通过路由机制分发name-address匹配信息



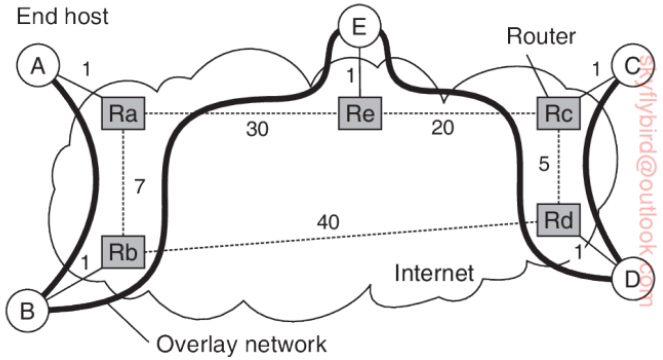
Overlay-based Multicast

- Overlay network, 覆盖网络
  - Communication path → virtual link
  - Multicast along the overlay
- Structure: tree or mesh
  - cost vs robustness





Overlay的性能/质量



- 三个性能指标
- Link stress:
  - counts how often a packet crosses the same link
- Stretch (relative delay penalty):
  - the ratio of the overlay network delay between two nodes over that of the underlaying network
- Tree cost:
  - the aggregated link costs

Gossip-based Multicast

- Data dissemination with local information only
- Propagation in a gossip way
  - Each node randomly choose destinations
- Simple but no guarantee of delivery
  - Only guarantee of “eventual delivery”
- Especially suitable for many-to-many

命名系统

- 无层次命名：解析是关键
  - 广播/多播、转发指针、宿主位置、DHT、搜索树
- 结构化命名：基于路径来访问
  - 名称空间/命名图
  - 名称服务器：迭代vs.递归
- 基于属性命名：目录服务
  - <属性，值>
  - 集中式
  - 层次化：LDAP
  - 去中心化

命名差别在哪

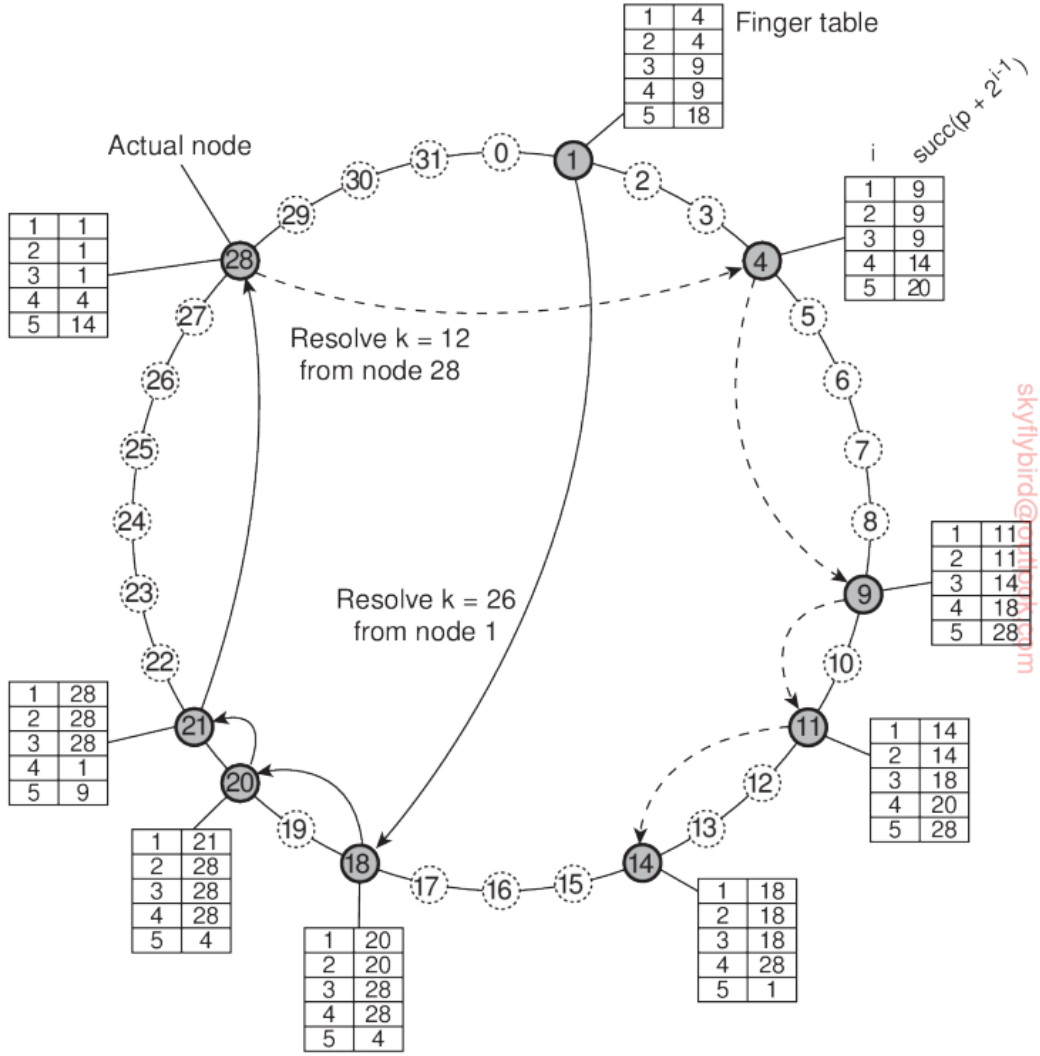
无结构命名

- Flat/Unstructured names
  - 命名只是单纯的标识符，可能是随机生成的串
  - 名字完全不反映位置/地址信息
  - 命名简单
- 关键问题：如何解析?
  - 简单方案：广播、转发指针
  - 宿主地址
  - 分布式哈希表
  - 分层定位

Chord

节点怎么加速，算法增加指针跳跃查找

- Chord结构：通过finger table提高搜索效率



- Each node p maintains a finger table $FT_p[]$ with at most m entries:

$$FT_p[i] = succ(p + 2^{i-1})$$

  Note: the i-th entry points to the first node succeeding p by at least 2^{i-1}.

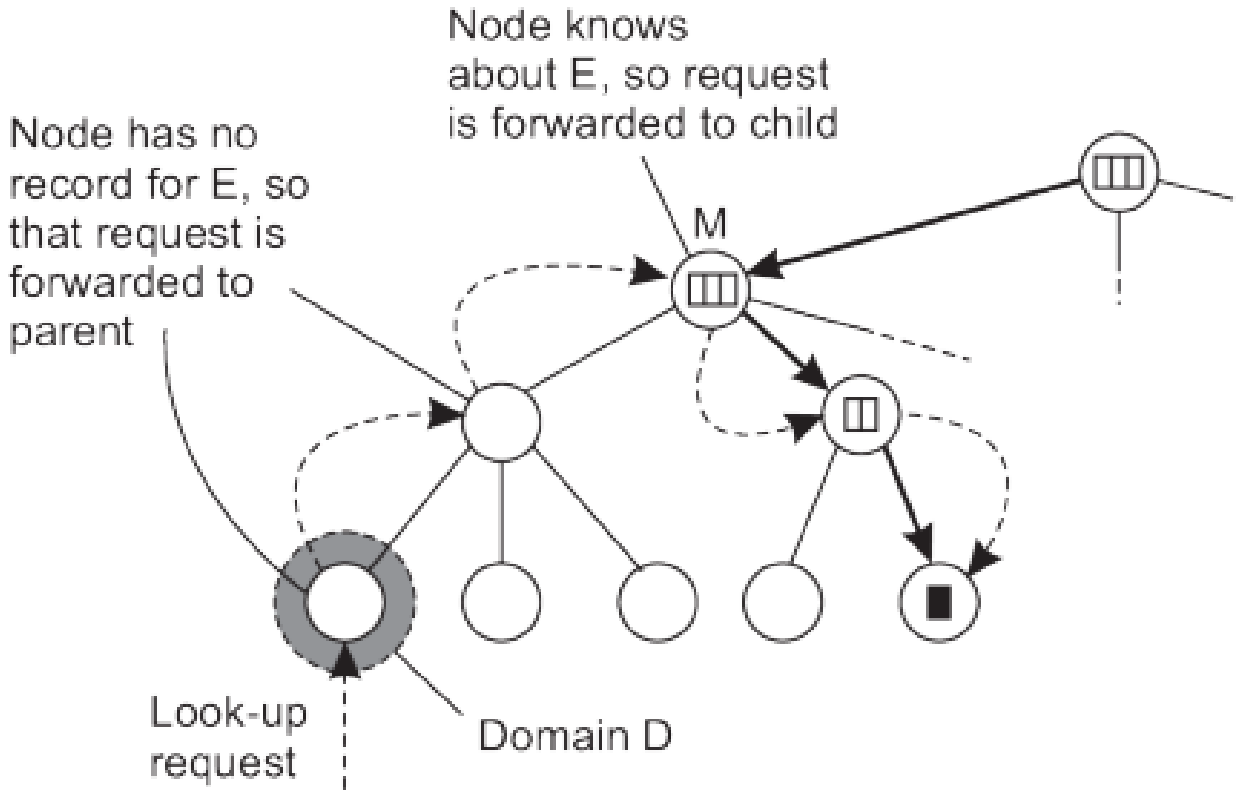
- To look up a key k, node p forwards the request to node with index j satisfying

$$q = FT_p[j] \le k < FT_p[j+1]$$

- If $p < k < FT_p[1]$, the request is also forwarded to $FT_p[1]$

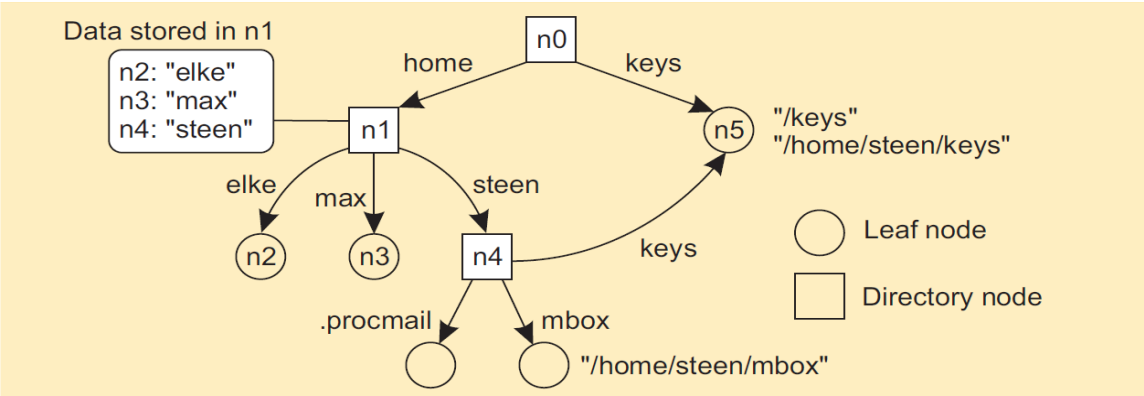
分层定位

- 搜索查找过程
  - 从叶子节点开始往上搜索，直到有节点知道目标实体
  - 然后沿此节点向下搜索到目标。

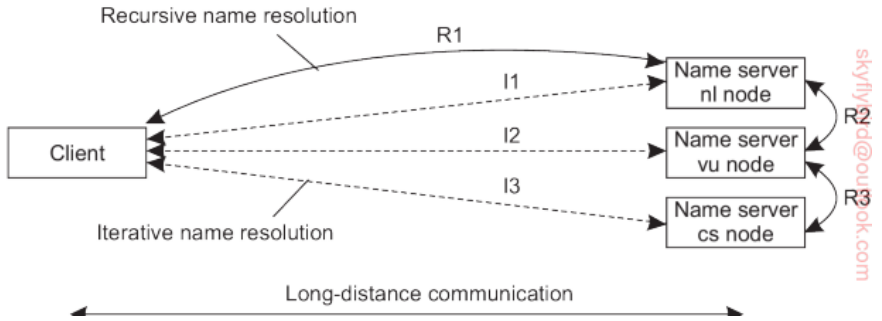


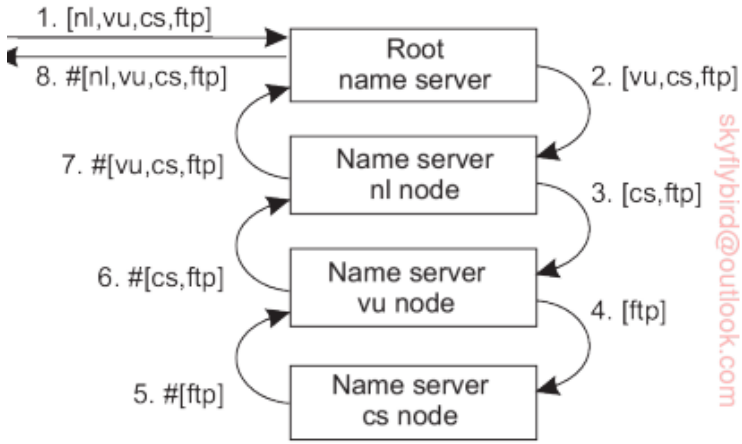
结构化命名

- 根据一定结构或者规则进行命名
- Human-readable names
- 如：
  - 基于目录结构的文件命名
  - 基于层次的主机名



- 递归式命名解析
  - 迭代式 vs. 递归式





基于属性的命名

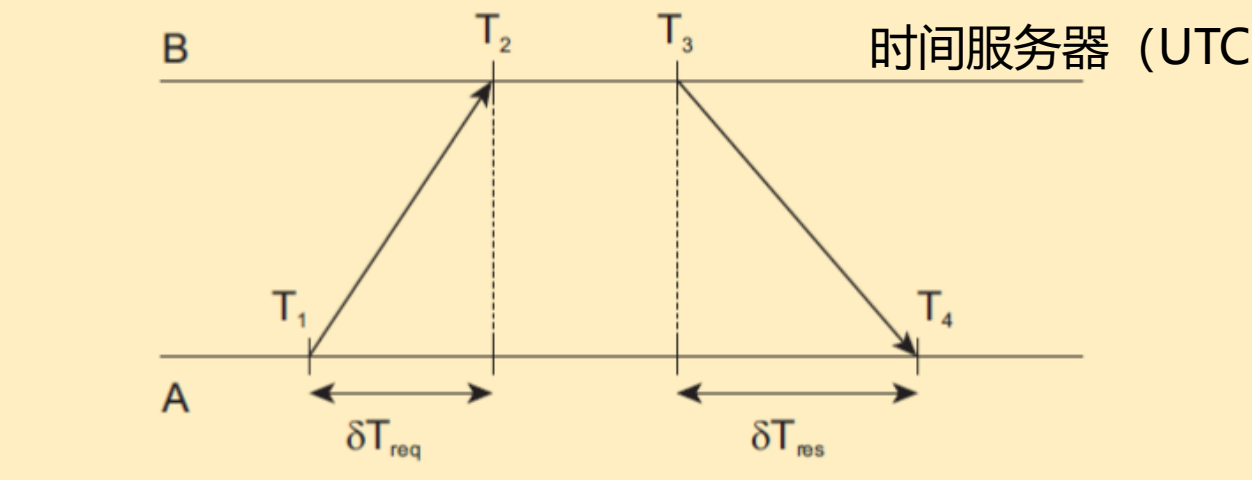
- 基于属性的命名也称为目录服务
- 需求：
  - 在很多场景中，通过实体的属性查询实体很方便
  - 如：传统电话号码目录簿、门户网站站点列表
- 问题/难点：
  - 查找操作代价高：需要进行属性值匹配和比对
- 属性构建（命名）
  - 定义属性、值
  - 基本需要手工设定
- 名称存储与查找
  - 目录服务器

协同与同步机制

- 同步是分布式系统的基本问题、核心问题
  - 时间 → 顺序
- 时钟同步：最根本的途径
  - 基于交换时钟值：UTC、GPS；NTP、Berkeley算法
- 顺序协同：更直接的途径
  - Happens-Before关系
  - Lamport逻辑时钟、向量时钟
- 进程同步：协作者控制、协调行为顺序
  - 互斥、选举
- 定位系统
  - 地理坐标、逻辑坐标

时钟同步

- 从一个时间服务器上获得当前的时间



如果快了作何处理?
如果慢了呢?

- 服务器之间的时间偏差和延迟为：

$$\delta T_{req} = T_2 - T_1 \approx T_4 - T_3 = \delta T_{res} \qquad \delta = \frac{(T_4 - T_1) - (T_3 - T_2)}{2}$$

$$\theta = T_3 + \frac{(T_2 - T_1) + (T_4 - T_3)}{2} - T_4 = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$$

NTP（Network time protocol）：收集多个（θ, δ），选择最小 δ 对应的θ

Lamport逻辑时钟

简单理解可背下来

Each process P_i maintains a local counter C_i and adjusts this counter

1. For each new event that takes place within P_i, C_i is incremented by 1.
2. Each time a message m is sent by process P_i, the message receives a timestamp $ts(m) = C_i$.
3. Whenever a message m is received by a process P_j, P_j adjusts its local counter C_j to $\max\{C_j, ts(m)\}$; then executes step 1 before passing m to the application.

Notes

- Property P1 is satisfied by (1); Property P2 by (2) and (3).
- It can still occur that two events happen at the same time. Avoid this by breaking ties through process IDs.

P1：如果 a 和 b是同一个进程中的两个事件，并且 a 在 b之前到达，则有：a->b
P2：如果 a 是消息的发送者，b是消息的接收者，则 a->b

Lamport逻辑时钟

Each process P_i maintains a local counter C_i and adjusts this counter

1. For each new event that takes place within P_i, C_i is incremented by 1.
2. Each time a message m is sent by process P_i, the message receives a timestamp $ts(m) = C_i$.
3. Whenever a message m is received by a process P_j, P_j adjusts its local counter C_j to $\max\{C_j, ts(m)\}$; then executes step 1 before passing m to the application.

Notes

- Property P1 is satisfied by (1); Property P2 by (2) and (3).
- It can still occur that two events happen at the same time. Avoid this by breaking ties through process IDs.

P1：如果 a 和 b是同一个进程中的两个事件，并且 a 在 b之前到达，则有：a->b
P2：如果 a 是消息的发送者，b是消息的接收者，则 a->b

全序多播（Total-ordered Multicast）

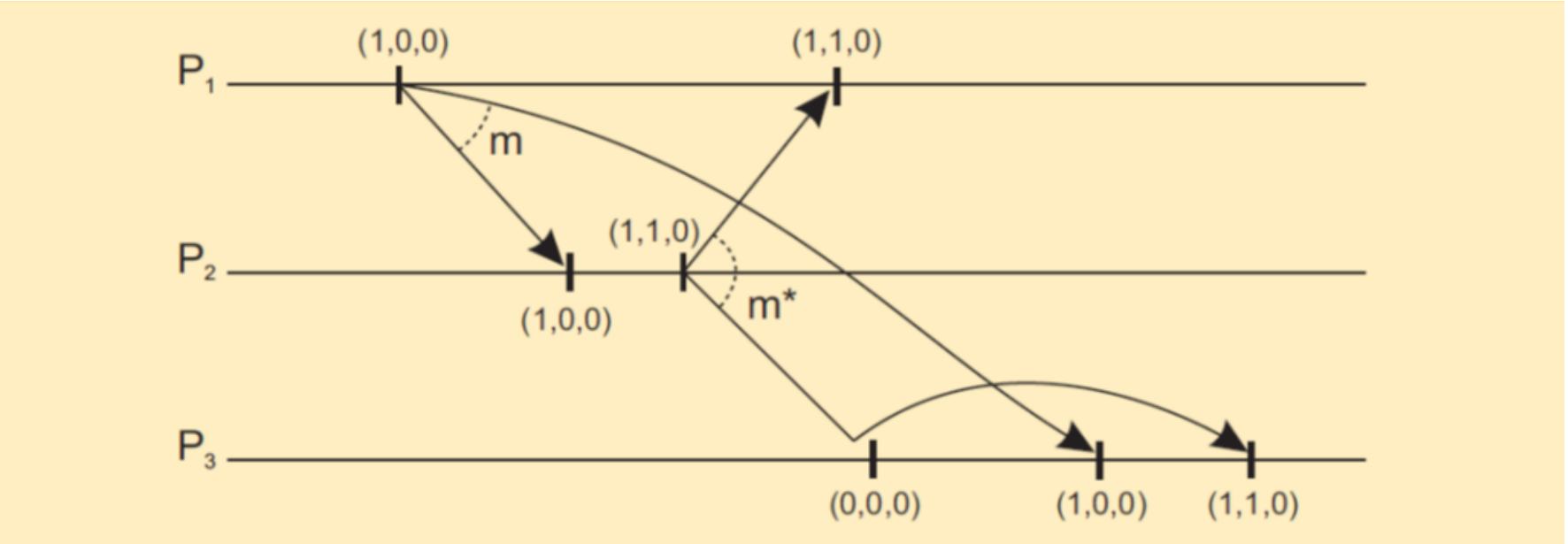
- 核心思路
  - 进程P_1将加上时间戳的消息m_i发送到所有的进程， 消息本身保存在本地的队列queue中；
  - 任何新到达P_j的消息放在队列$queue_j$中，根据它的时间戳进行排序，并向所有进程广播确认消息；
- P_j在满足如下条件时会将消息m_i交付给应用程序：
  - m_i是队列$queue_j$的队列头；
  - 对每个进程P_k，$queue_j$ 中有一个m_i 的Ack确认消息。

  （可以替换为：在队列$queue_j$存在一个消息m_k具有更大的时间戳）

注意：假定通信是可靠的，并且是FIFO排序的。

因果有序多播

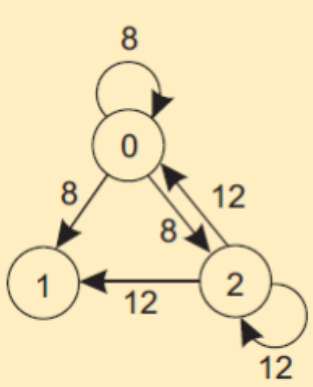
- 强制因果有序的通信



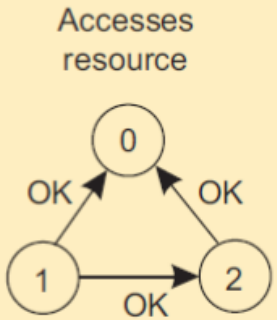
– 当P_3收到 m*来自P_2，需要推迟“接受”，直到收到来自P_1的m

Ricart & Agrawala互斥算法

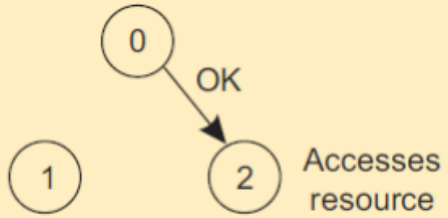
- 三个进程例子



(a)



(b)

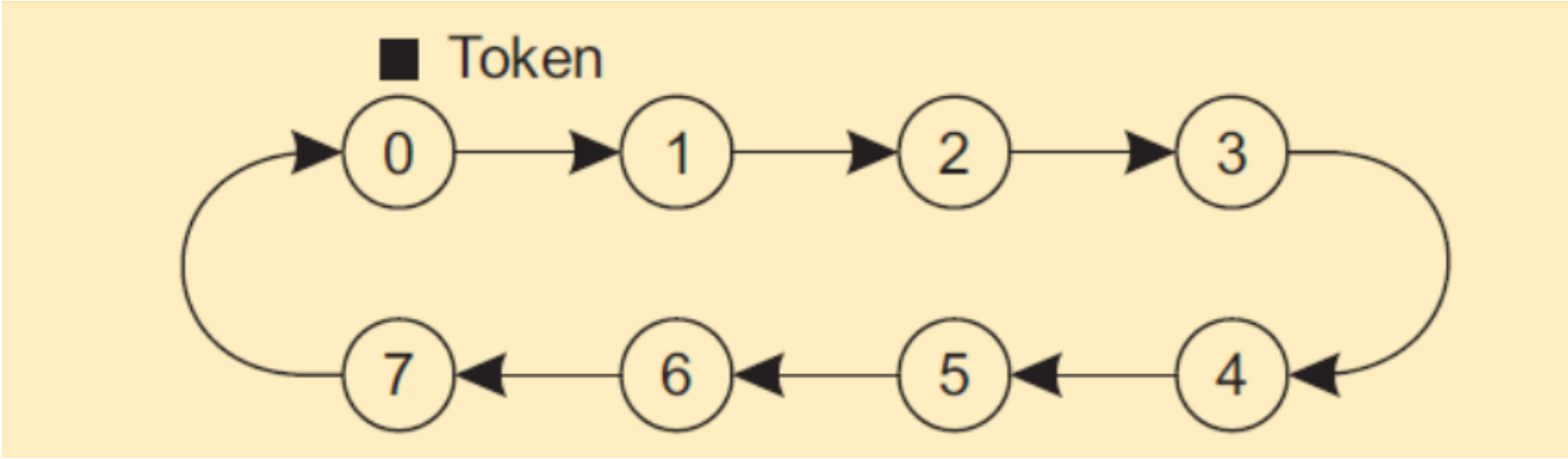


(c)

(a) Two processes want to access a shared resource at the same moment.
(b) P_0 has the lowest timestamp, so it wins.
(c) When process P_0 is done, it sends an *OK* also, so P_2 can now go ahead.

令牌环算法

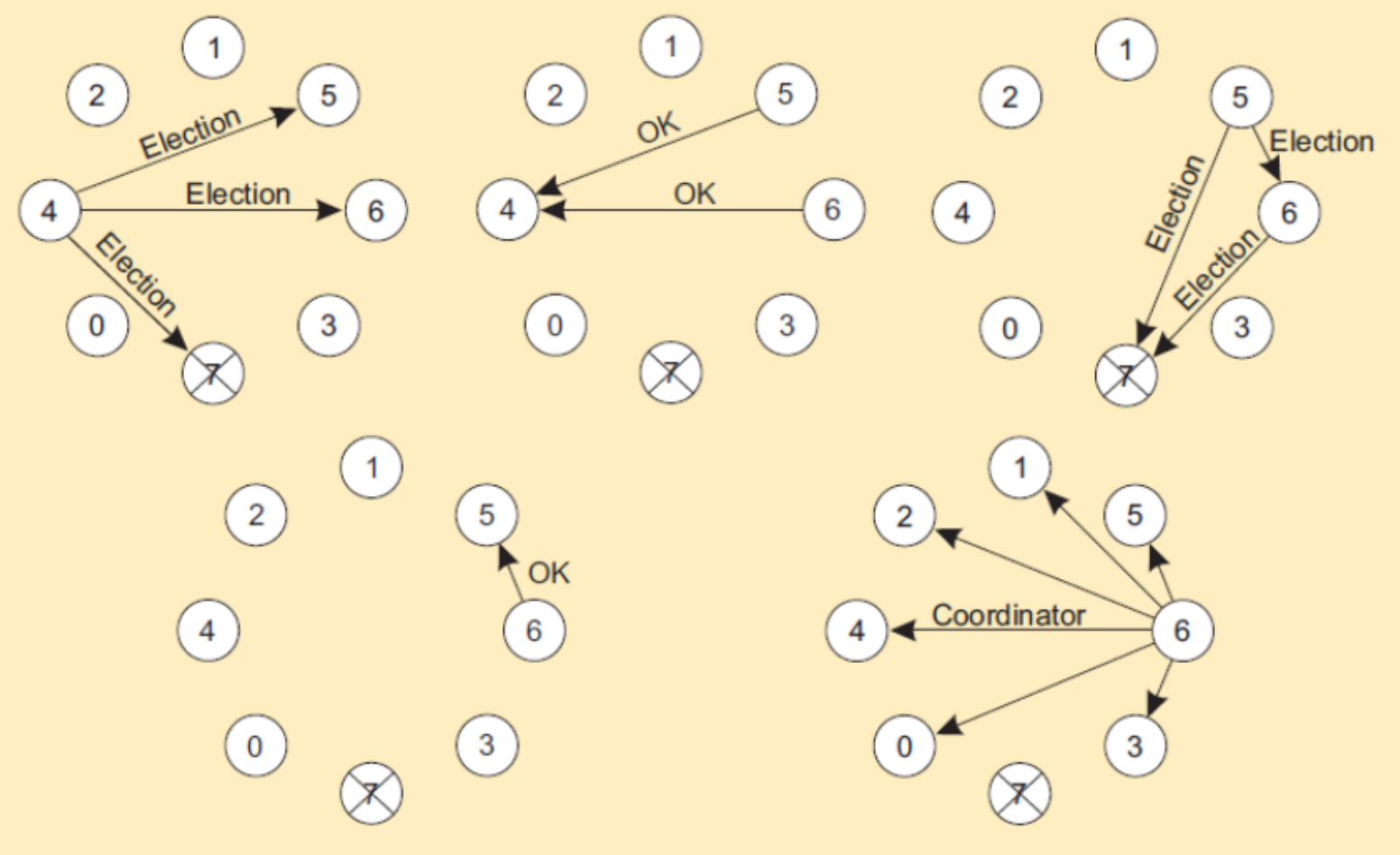
- 要点：
  - 将进程组织成逻辑环，令牌在这些进程之间传递。
  - 拥有令牌的进程允许进入临界区。
- 覆盖网络构成一个逻辑环



互斥算法比较

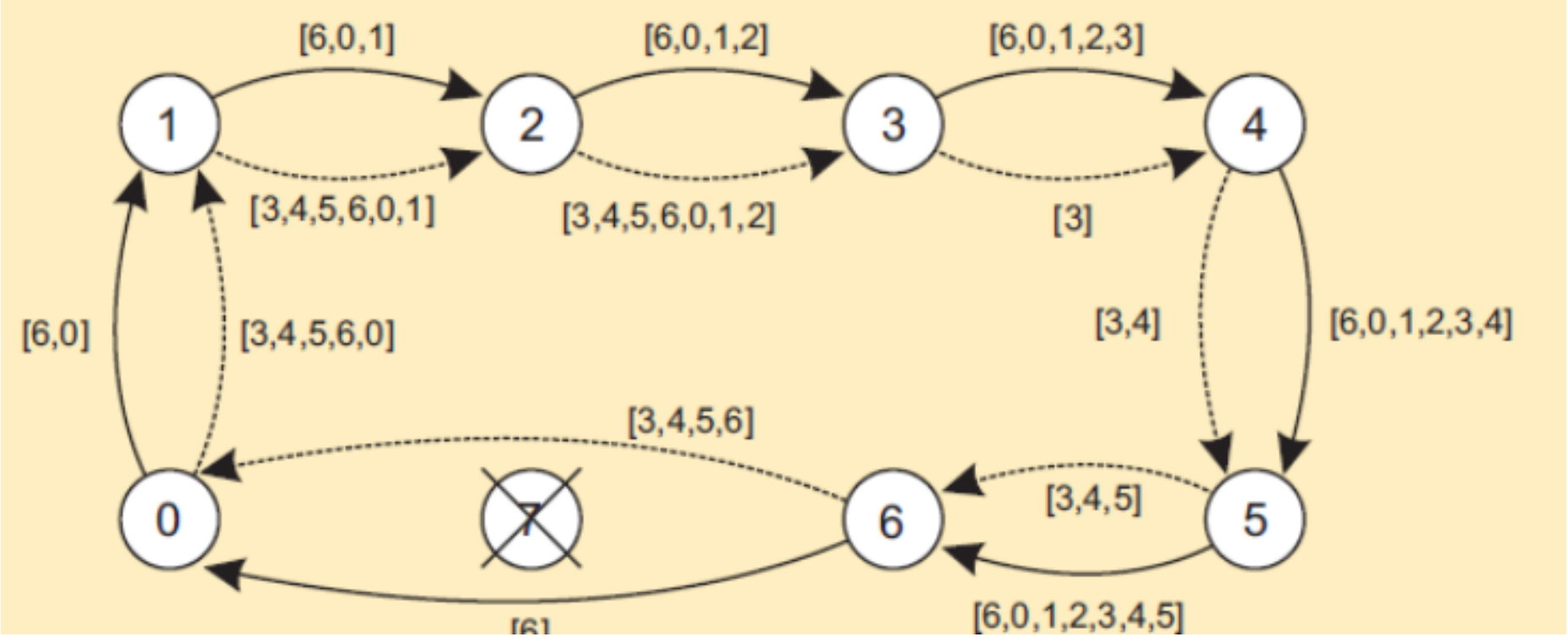
Algorithm	Messages per entry/exit	Delay before entry (in message times)
Centralized	3	2
Distributed	$2 \cdot (N-1)$	$2 \cdot (N-1)$
Token ring	$1, \ldots, \infty$	$0, \ldots, N-1$
Decentralized	$2 \cdot m \cdot k + m, k = 1, 2, \ldots$	$2 \cdot m \cdot k$

Bully算法



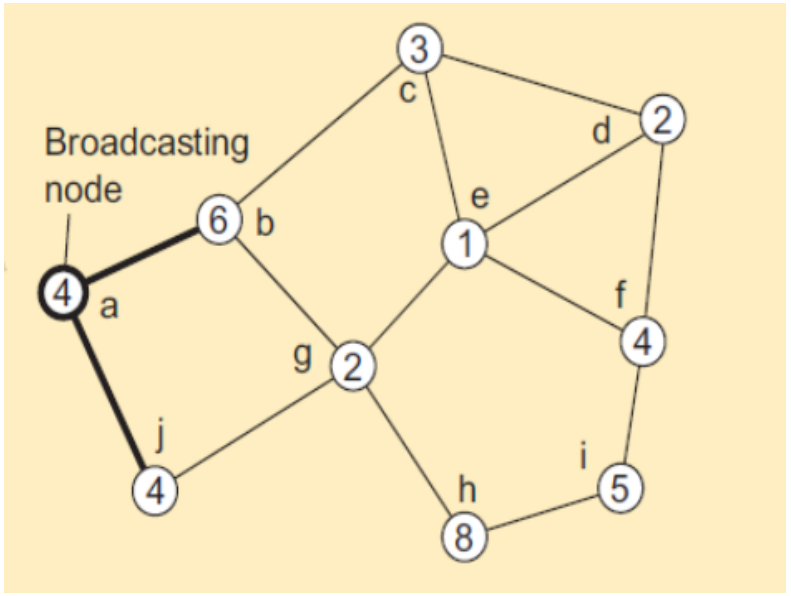
(a) Process 4 holds an election.
(b) (b) Processes 5 and 6 respond, telling 4 to stop.
(c) Now 5 and 6 each hold an election. (d) Process 6 tells 5 to stop.
(e) Process 6 wins and tells everyone.

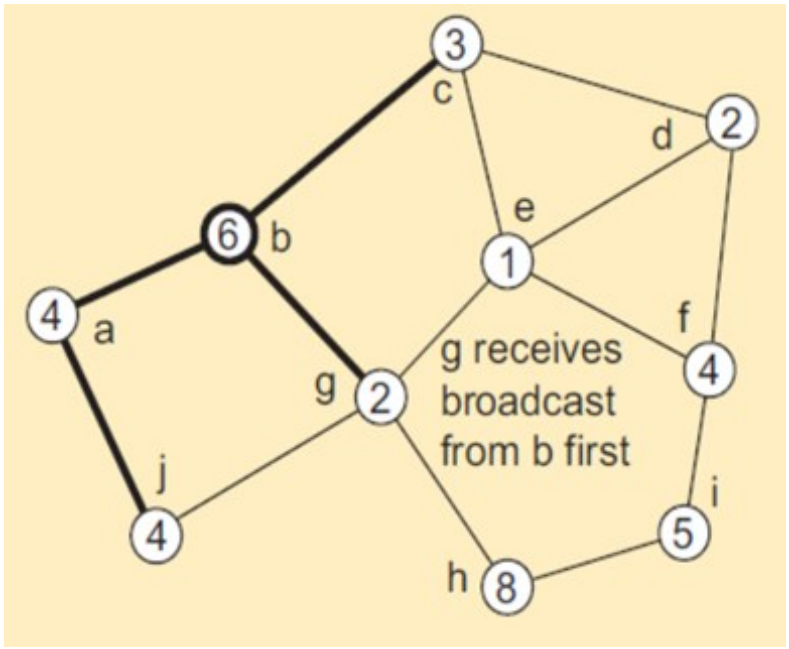
Ring算法

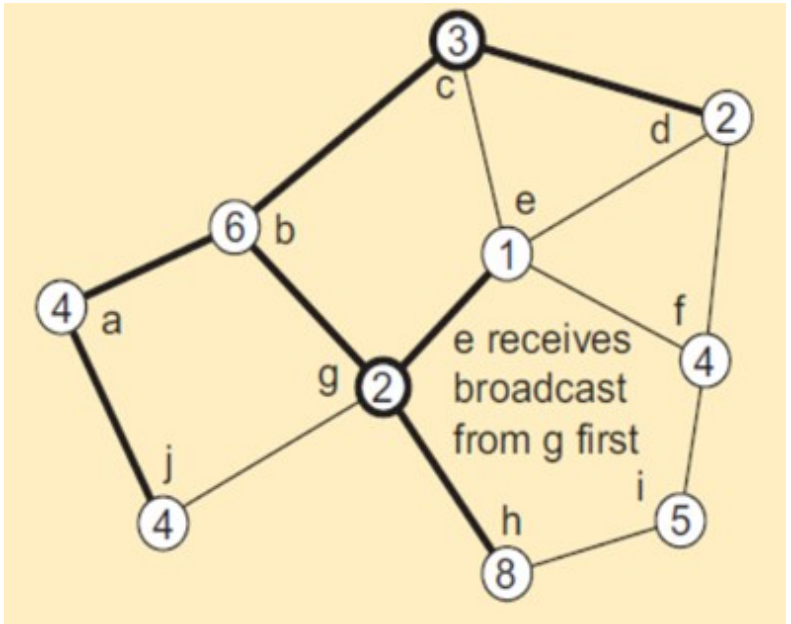


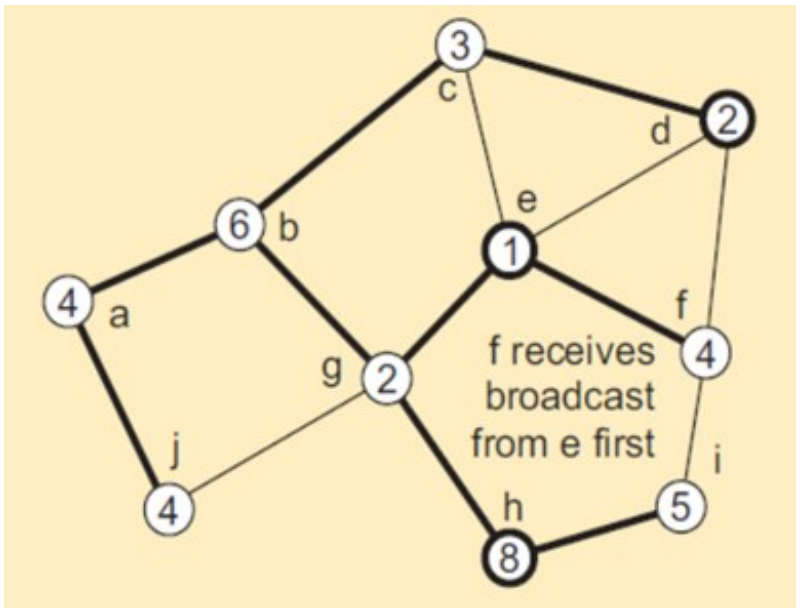
The solid line shows the election messages initiated by P6;
The dashed one those by P3.

无线环境中的选举


Broadcasting
node
3
c
2
d
6
b
e
1
4
a
f
4
g
2
j
4
i
5
h
8


3
c
d
2
6
b
e
1
4
a
f
4
g
2
g receives
broadcast
from b first
j
4
i
5
h
8

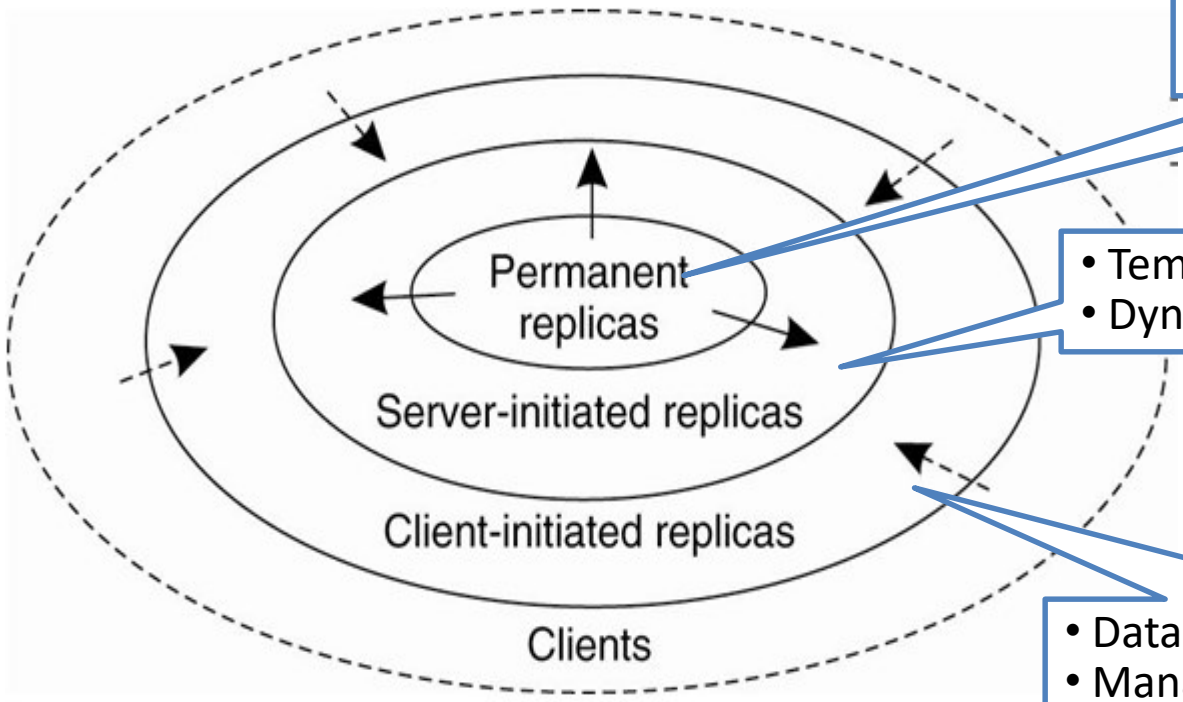

3
c
d
2
e
6
b
1
4
a
f
4
g
2
e receives
broadcast
from g first
j
4
i
5
h
8


3
c
d
2
e
6
b
1
4
a
f
4
g
2
f receives
broadcast
from e first
j
4
i
5
h
8

副本与一致性

- Replica management
  - 副本/缓存类型；副本放置、内容分发
- Consistency models
  - 数据为中心的一致性（读写并重系统）
    - 连续（持续） 一致性、操作一致性
  - 用户为中心的一致性（读为主）
    - 单调读、 单调写、 读写一致（写后读）、 写读一致（读后写）
- Consistency protocols
  - 数据为中心：
    - 持续一致性协议： 更新扩散过程限制偏差
    - 操作一致性协议： 主备份写、 全复制写、 多数（Quorum） 写
  - 用户为中心：读写集检查

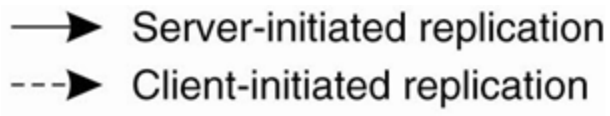
Data Content Placement



- Initial (Seed) replicas
- Statically configured

- Temporary replicas
- Dynamically placed by server

- Data “caches”
- Managed by clients



Content Distribution

- How to distribute
  - Push: suitable for permanent/server-initiated replicas
    - High read-to-update ratio
  - Pull: suitable for caches
    - Low read-to-update ratio

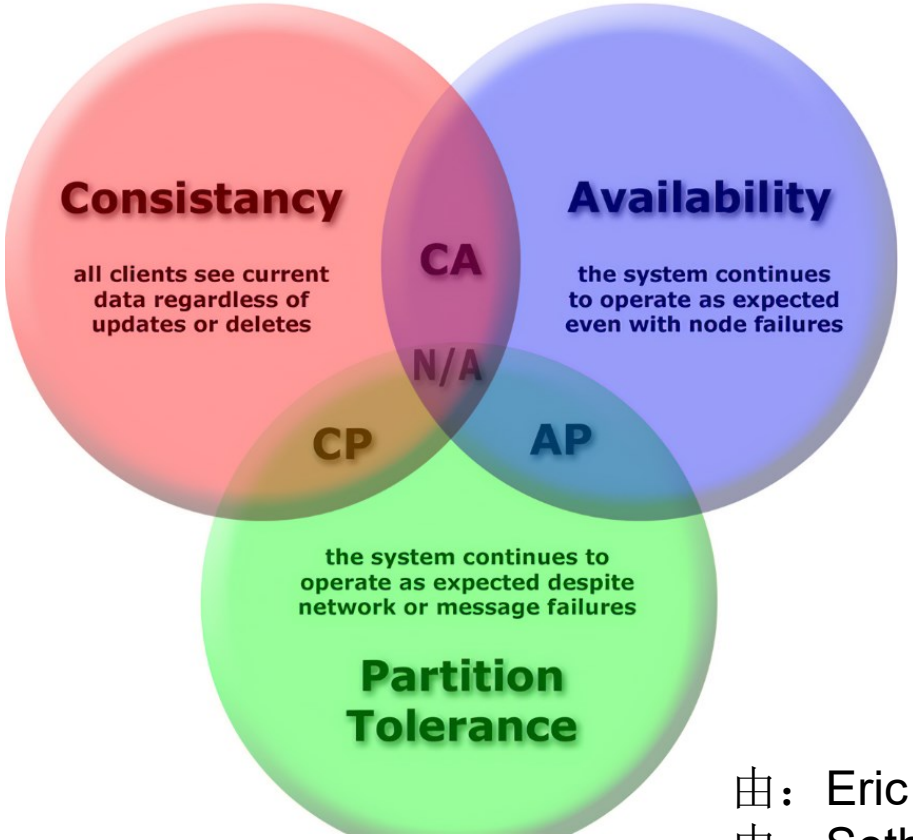
Issue	Push-based	Pull-based
State at server	List of client replicas and caches	None
Messages sent	Update (and possibly fetch update later)	Poll and update
Response time at client	Immediate (or fetch-update time)	Fetch-update time

Consistency Models

- Consistency Model
  - A contract between processes and the data store.
  - It says that: if processes obey certain *rules*, the store promises to work *correctly*.
- "Correctness" :
  - If a process performs a read operation on a data item, the operation should return a value that shows the results of the last write operation on that data.
- Which is the last write, without global clock?
  - Consistency model defines the values that a read operation can return.
  - Tradeoff between *consistency level* and *maintenance cost*

CAP理论

- 指一个分布式系统中 CAP三者不可得兼



①**一致性**：客户端的读操作要么读到最新的数据，要么读取失败。

②**可用性**：任何客户端的请求都能得到响应数据。

③**分区容忍性**：当消息丢失或延迟到达时，系统仍会继续提供服务，不会挂掉。





由：Eric A. Brewer 在PODC 2000特邀报告中提出

由：Seth Gilbert, Nancy Lynch 正式证明

Seth Gilbert, Nancy Lynch, Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services, ACM SIGACT News 33(2), June 2002 pp 51–59.

Consistency Models

- Data-centric consistency models
  - Continuous consistency (on data content)
  - Update order consistency (on data operations)
    - Sequential consistency
    - Causal consistency
    - Grouping operations
- Client-centric consistency models
  - Monotonic reads
  - Monotonic writes
  - Read your writes
  - Writes follow reads

Sequential Consistency

- No "time" in the definition of sequential consistency model
- A operation sequence is *valid* provided that *all processes see the same sequence.*

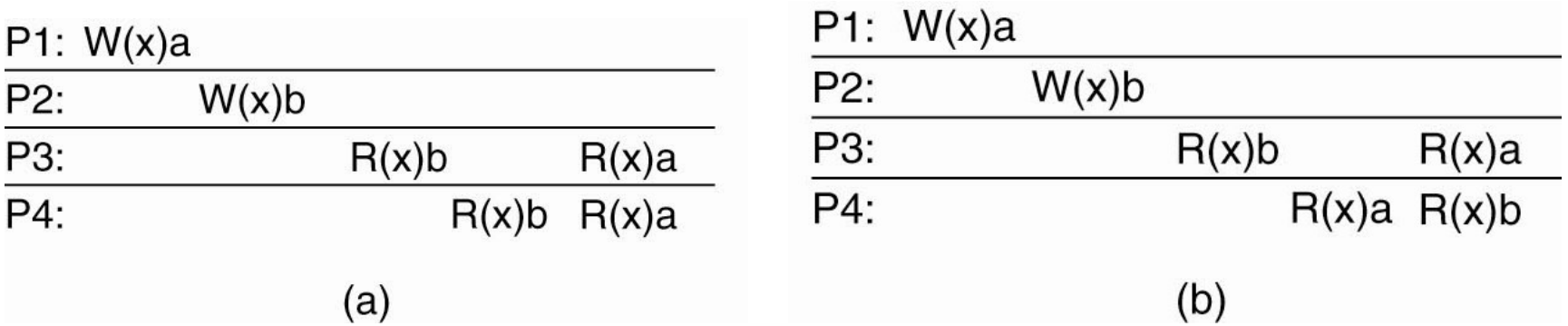
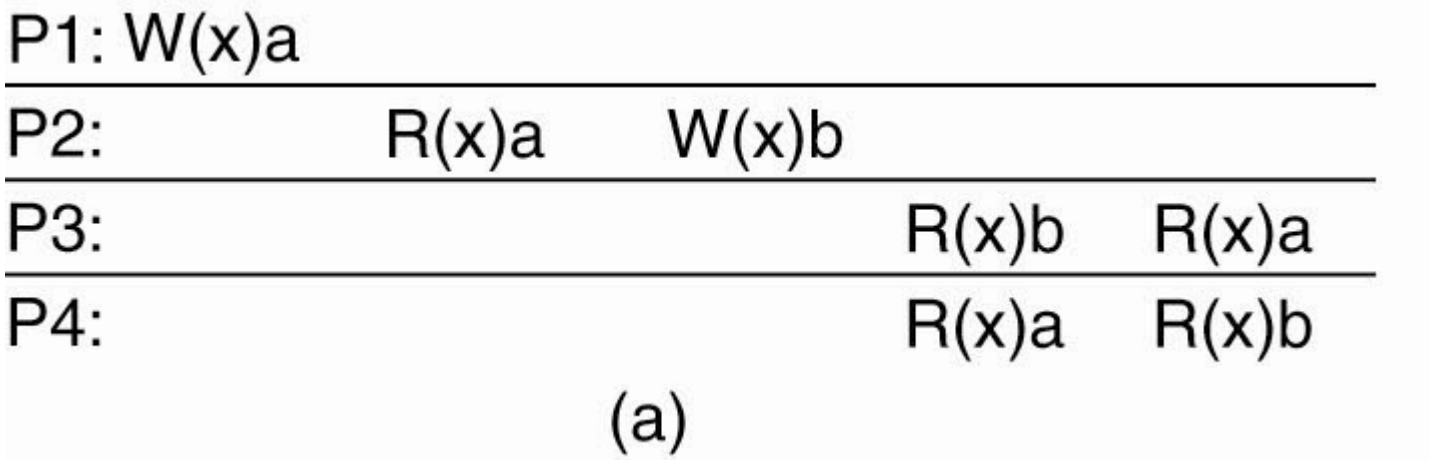
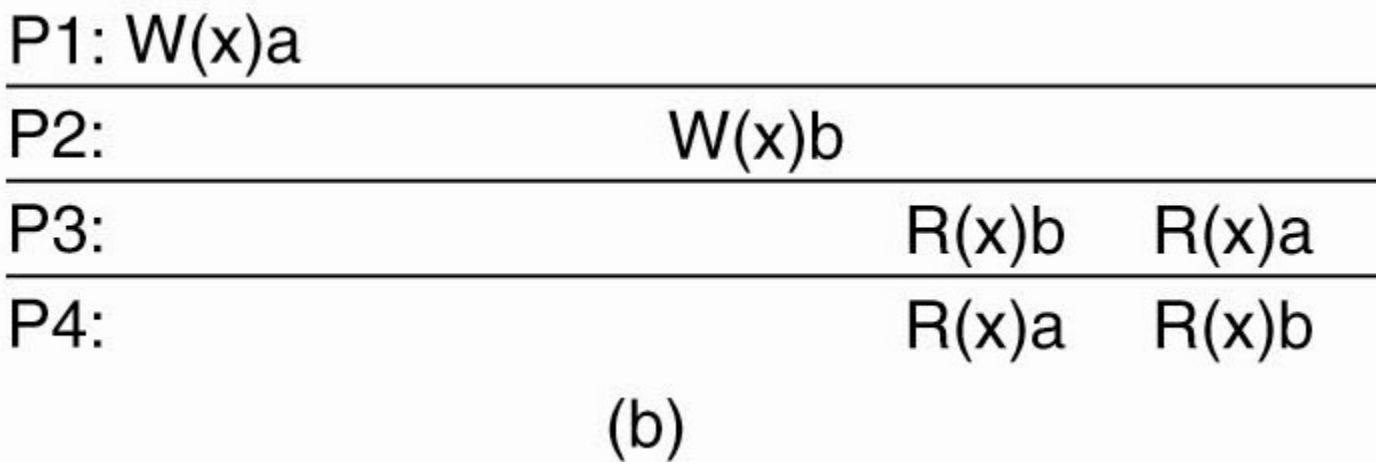


Figure 7-5. (a) A sequentially consistent data store.
(b) A NOT sequentially consistent data store.

Causal Consistency



(a)



(b)

Is the operation sequence causally consistent?

连续一致性：限定数值偏差

Problem

We need to ensure that $v(t) - v_i < \delta_i$ for every server S_i.

Approach

Let every server S_k maintain a view $TW_k[i,j]$ of what it believes is the value of $TW[i,j]$. This information can be gossiped when an update is propagated.

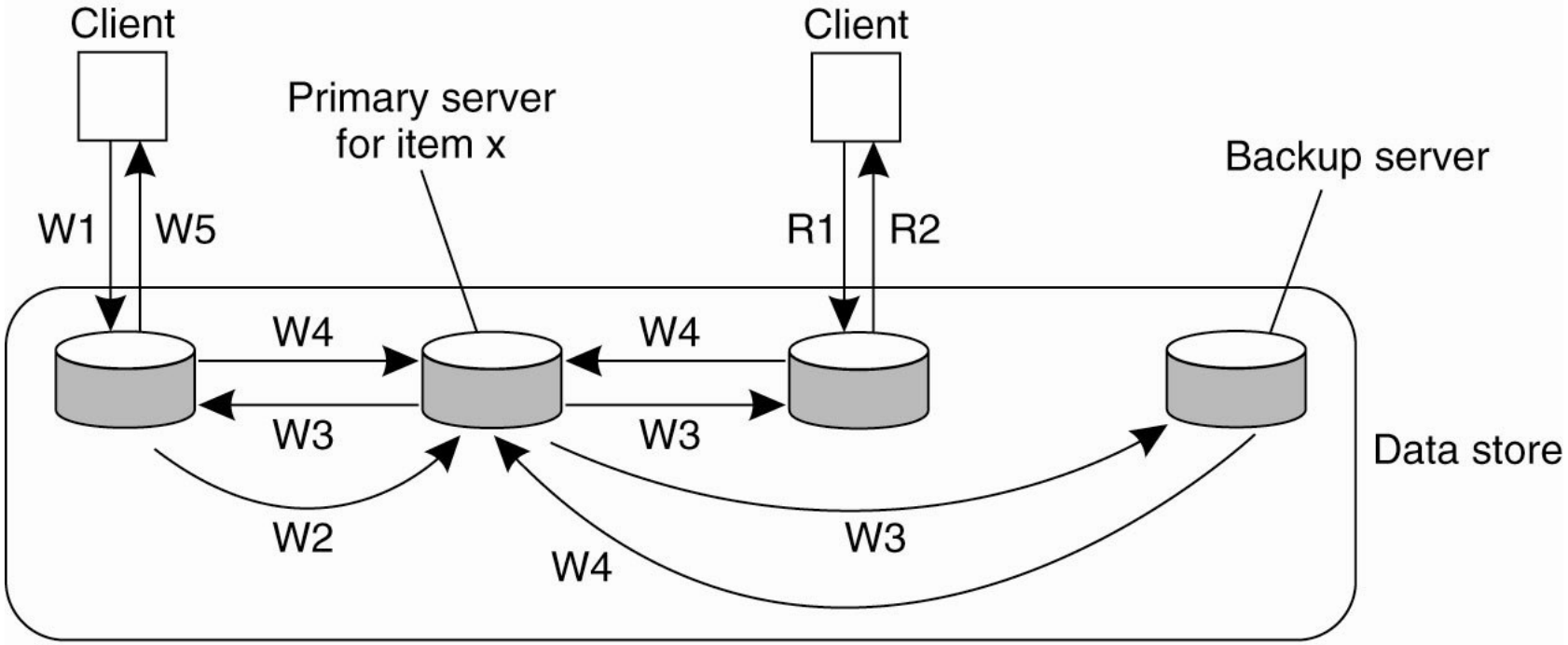
Note

$$0 \leq TW_k[i,j] \leq TW[i,j] \leq TW[j,j]$$

- 基本操作
  - S_i 扩散写操作 originating from S_j to S_k（写操作传播机制）
  - S_k会获得$TW[i,j]$，如发现更新操作步调不一致，把写操作从日志中转发给S_i
  - 转发操作可以把S_k的视图$TW_k[i,k]$往$TW[i,k]$靠近
  - 当应用程序提交一个新的写操作时，S_k会把其视图往$TW[k,k]$推，从而导致

    $$TW[k,k] - TW_k[i,k] > \frac{\delta_i}{N-1}$$
  - 但是本方法能够确保$TW[i,k] - TW_k[i,k] \leq \delta_i$

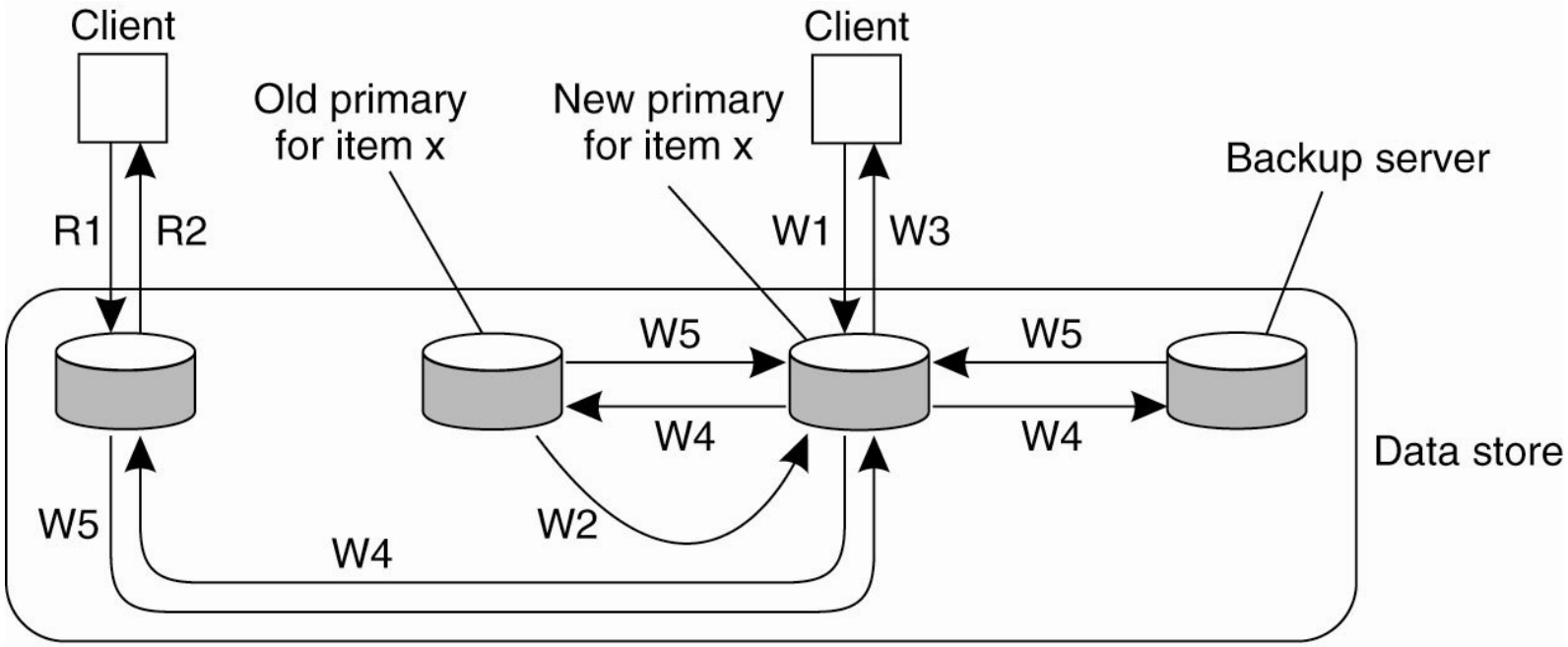
主副本协议：Remote-write Protocol



W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

R1. Read request
R2. Response to read

主副本协议：Local-write Protocol

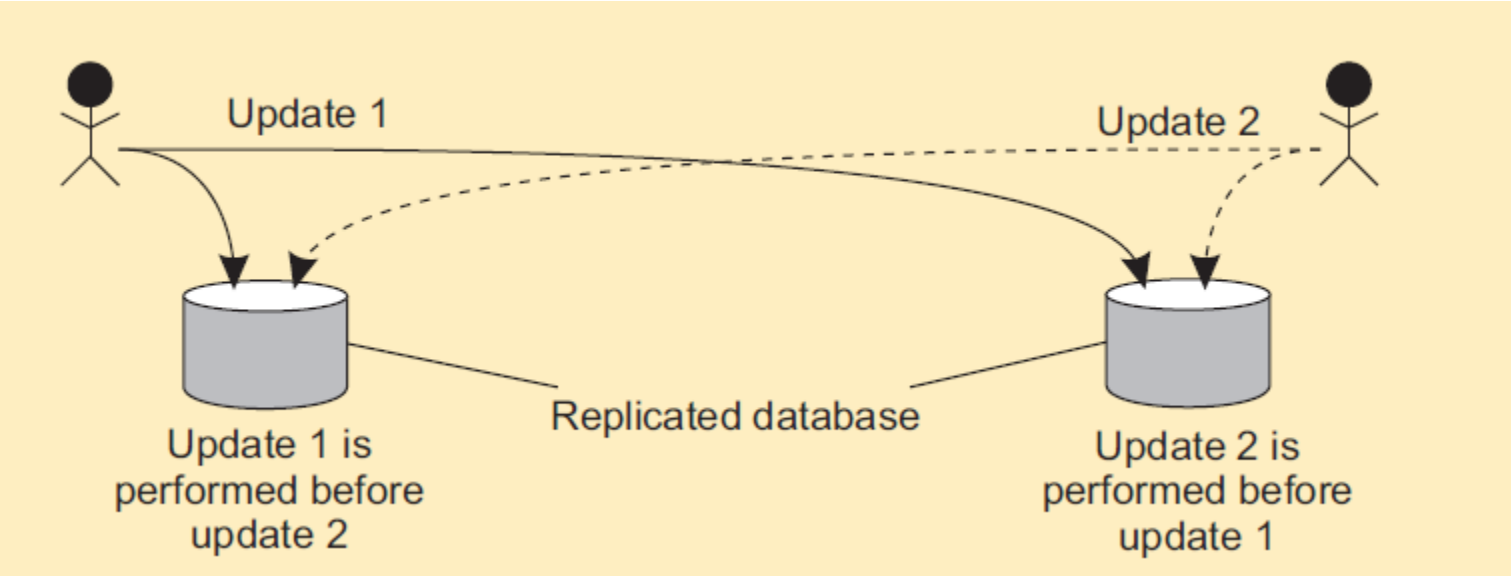


W1. Write request
W2. Move item x to new primary
W3. Acknowledge write completed
W4. Tell backups to update
W5. Acknowledge update

R1. Read request
R2. Response to read

复制写协议：Active Replication Protocol

- The operation is forwarded to all replicas
- Operations to be carried out in the same order everywhere
- Requires totally ordered multicasts using either Lamport timestamps (e.g.) or a central coordinator.



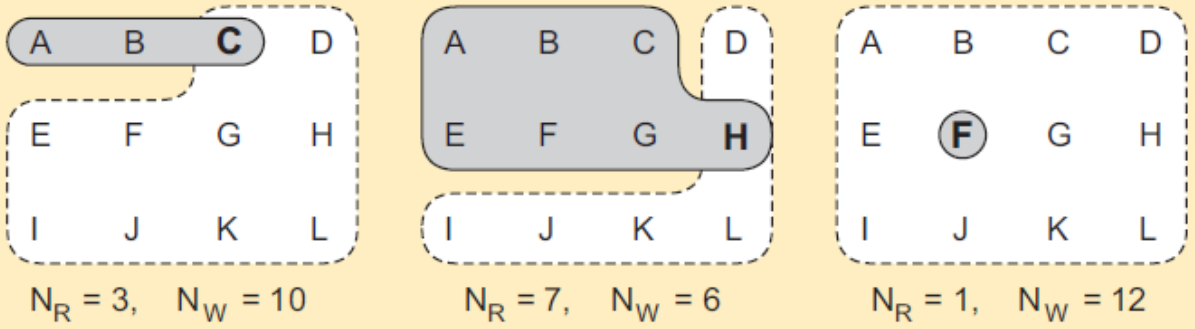
复制写协议：Quorum-Based Protocol

Quorum-based protocols

Ensure that each operation is carried out in such a way that a majority vote is established: distinguish read quorum and write quorum

Three examples of the voting algorithm. (a) A correct choice of read and write set. (b) A choice that may lead to write-write conflicts. (c) A correct choice, known as ROWA (read one, write all)

1. $N_R + N_W > N$
2. $N_W > N/2$



客户为中心的一致性协议

- 客户检查自己的更新是否完成
  - 每个写操作被唯一标识；
  - 每个客户维护两个集合：

    读操作集{客户的读操作相关的写操作}；

    写操作集{客户的写操作}
  - 单调读/写：

    相关的读/写操作集与读/写请求一起发送服务器；

    执行读操作前检查是否所有写已经在本地执行；

    没有的话：联系其他服务器进行更新，或转发读请求出去。
  - 读写一致（写后读）：读之前检查写操作集。
  - 写读一致（读后写）：写之前检查读操作集，并将读操作集加入写操作集。

容错技术

- 冗余容错（最基本方法）：容错进程组
- 共识Consensus
  - 系统模型假设：同步、异步；故障模型：Crash、Byzantine
  - 共识协议：Paxos、BFT等
- 可靠C-S通信
  - 可靠RPC
  - 可靠组通信（多播）：Scalability、Atomicity、Group membership
- 系统恢复
  - 检查点机制、消息日志

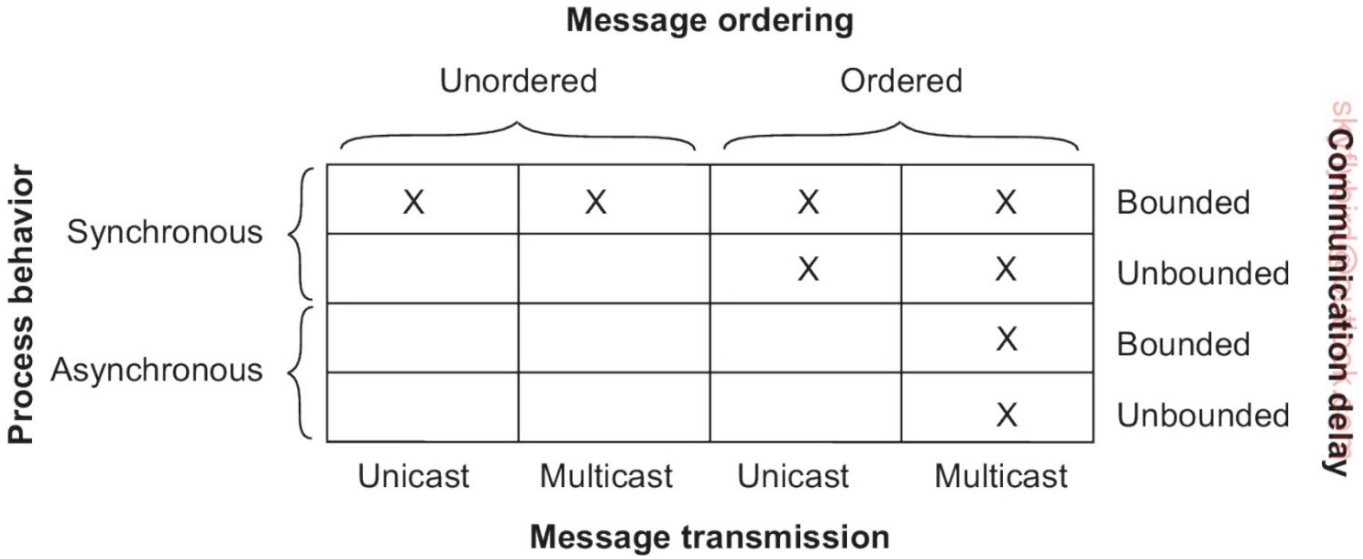
进程复制与故障掩盖（masking）

- 容错度：K-容错组
  - 一个进程组可以屏蔽最多 K 个成员失效
- 复制进程组的容错度：
  - 只是停止失效（crash/omission）：
    - 需要 K+1 成员保证正确输出；
  - 任意行为的失效：
    - 需要 2k + 1个成员才能保证正确输出：
    - 最坏情况：k个进程失效，而且给出相同的错误结果
    - K+1个正确进程可以输出正确结果
      - Client可以通过“相信多数”得到正确值
  - 上述结果的前提：
    - 所有正确进程行动一致（consensus）：相同顺序执行请求
      - 无失效的话：完全有序多播、集中式的排序器等

分布式共识（Consensus）

- 什么环境下能容忍节点失效达成共识?

M.J. Fischer, N.A. Lynch, and M.S. Paterson.
Impossibility of distributed consensus with one faulty process. J. ACM, 32(2):374–382, 1985.



Process synchrony: whether the processes operate in a lock-step mode. (There should be some constant c ≥ 1, such that if any process has taken c + 1 steps, every other process has taken at least 1 step.)

Delay bound: whether messages are delivered with a globally and predetermined maximum time.

Message ordering: whether messages are delivered in the order of sending in real global time.

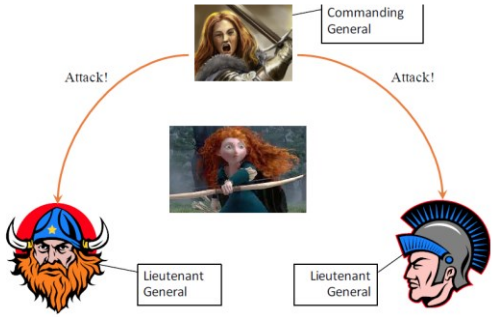
Transmission mode: unicasting or multicasting.

共识协议分类

泛洪　Paxos　Raft　ZAB　BFT　PBFT　PoW

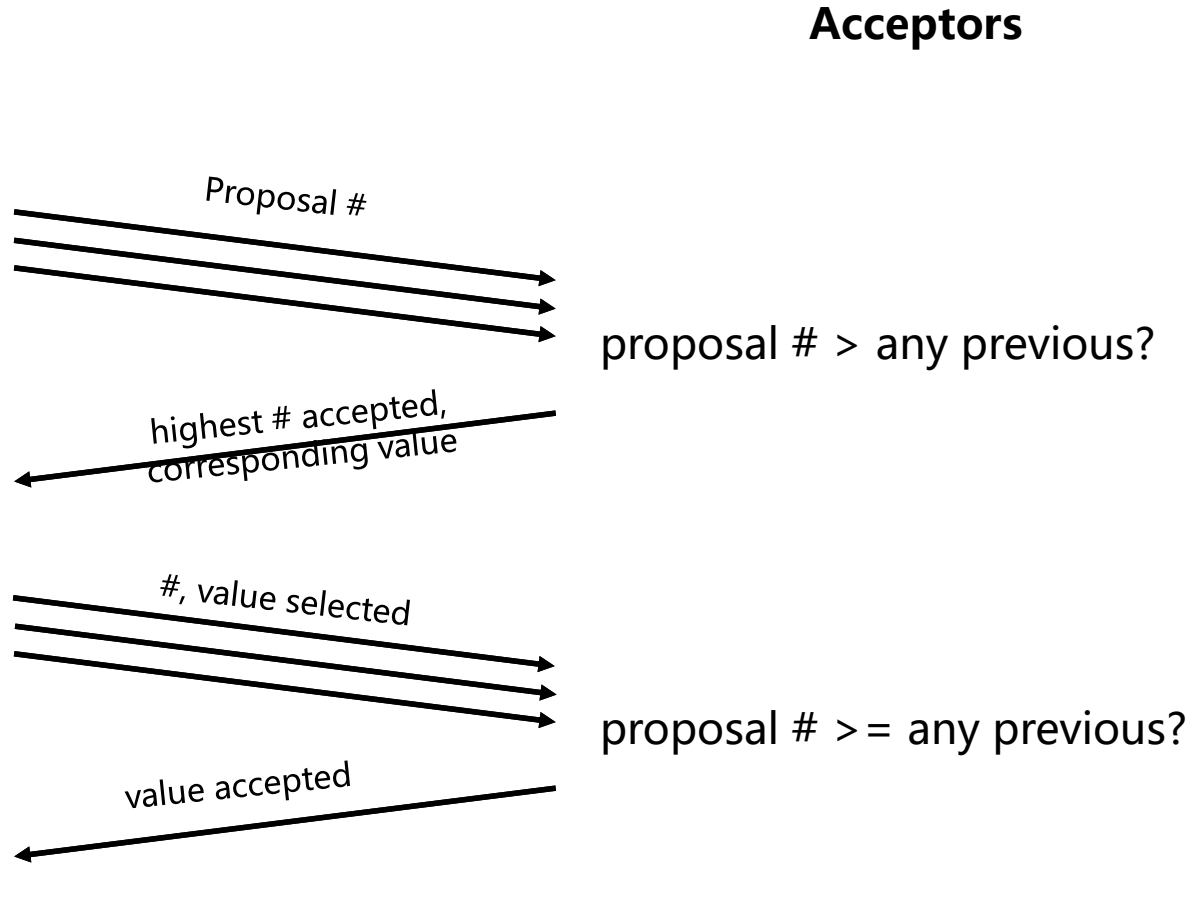
准确检测失效　能最终检测失效　拜占庭容错
（有恶意行为）

仅“停止失效”容错



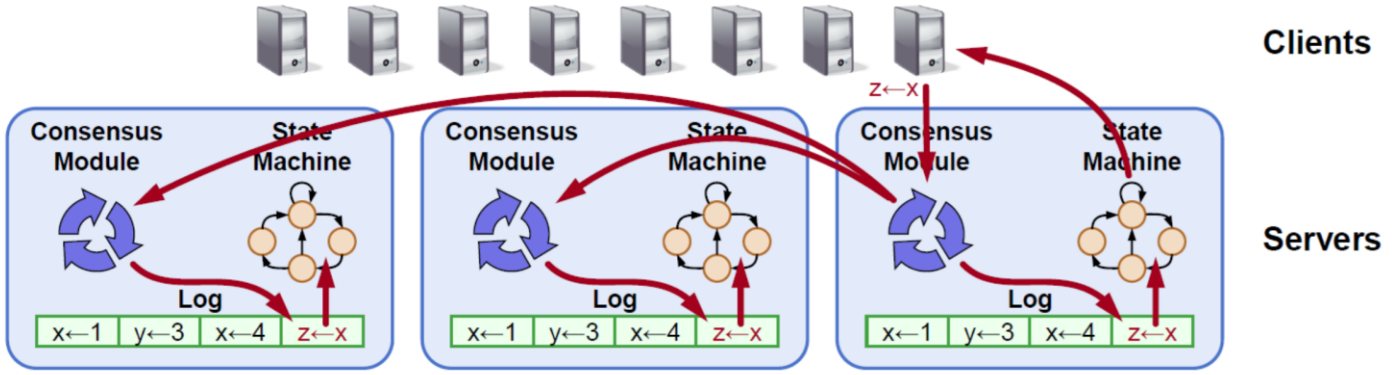
拜占庭共识问题（Byzantine agreement)
--L. Lamport, 1982

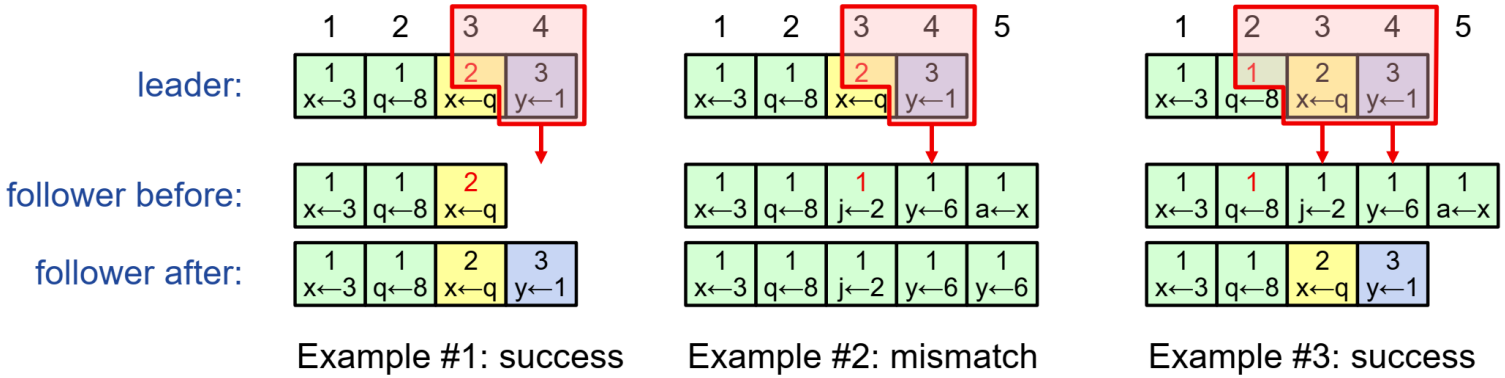
Paxos协议


Proposers
Acceptors
Choose unique proposal #
Proposal #
proposal # > any previous?
highest # accepted, corresponding value
Majority?
Select value for highest proposal # returned;
If none, choose own value
#, value selected
proposal # >= any previous?
value accepted
Majority? Value decided

Raft协议

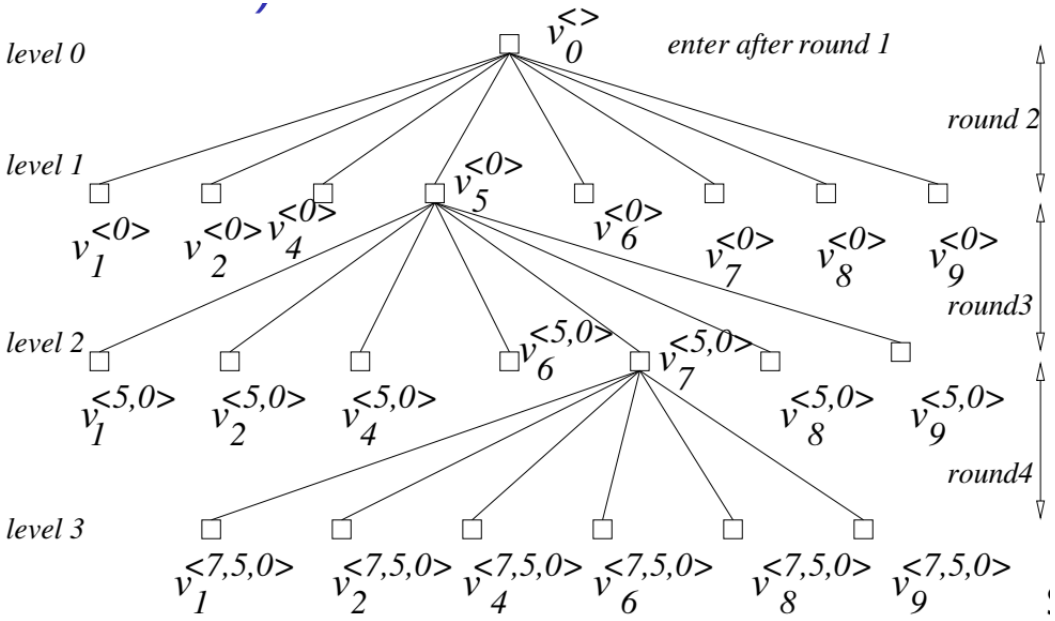
- AppendEntries include <index, term>of entry preceding new one
- Follower must contain matching entry;
- Implements an induction step, ensures Log Matching Property





BFT协议

- n=10, f=3，发起节点P0，当前节点P3



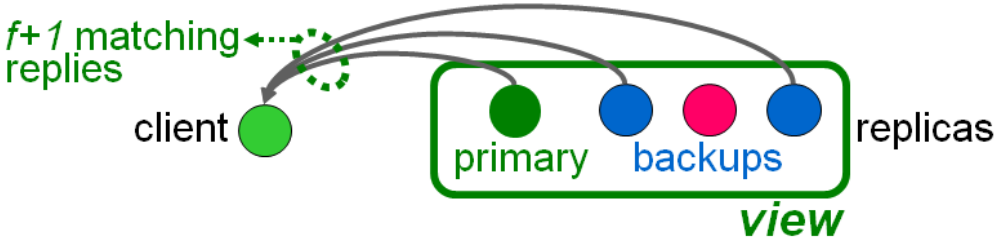
(round 1) P_0 sends its value to all other processes using *Oral Msg*(3), including to P_3.
(round 2) P_3 sends 8 messages to others (excl. P_0 and P_3) using *Oral Msg*(2).
P_3 also receives 8 messages.
(round 3) P_3 sends 8 × 7 = 56 messages to all others using *Oral Msg*(1);
P_3 also receives 56 messages.
(round 4) P_3 sends 56 × 6 = 336 messages to all others using *Oral Msg*(0);
P_3 also receives 336 messages.
The received values are used as estimates of the majority function at this level of recursion.

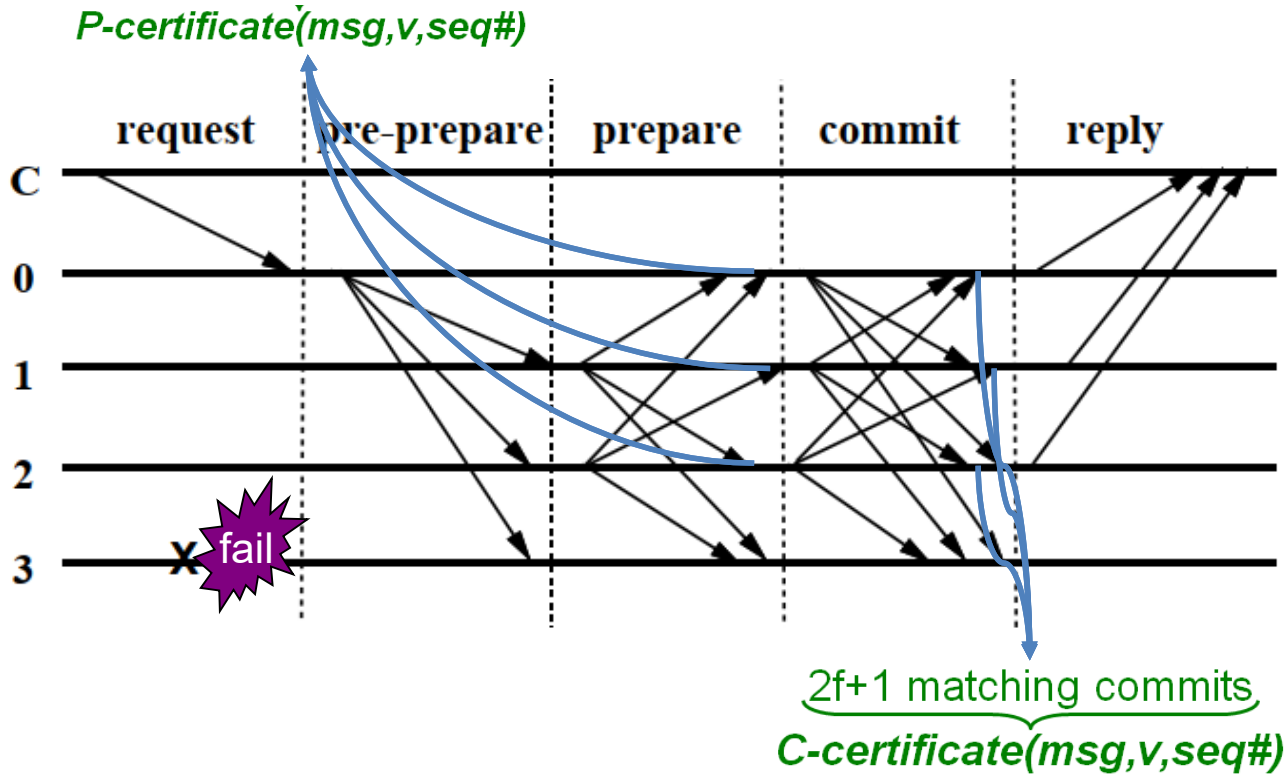
PBFT协议

Commit among all replicas

Commit message

{COMMIT,view,seq#,msg_digest,i}





消息容错

- 消息交付是否“遵循”某种顺序
- 单个接收者视角看
  - 遵循某种“客观存在/要求”的顺序
- 全局视角看
  - 不同接收者是否按相同的顺序
- FIFO顺序（单接收者视角）
  - 同一进程发的消息按“发送顺序”交付
  - 不同进程发的消息可以按不同顺序交付

Multicast	Basic message ordering	TO delivery?
Reliable multicast	None	No
FIFO multicast	FIFO-ordered delivery	No
Causal multicast	Causal-ordered delivery	No
Atomic multicast	None	Yes
FIFO atomic multicast	FIFO-ordered delivery	Yes
Causal atomic multicast	Causal-ordered delivery	Yes

可靠RPC

- RPC的失效处理
  - 重发消息、重做操作

1. The client is unable to locate the server.
2. The request message from the client to the server is lost.
3. The server crashes after receiving a request.
4. The reply message from the server to the client is lost.
5. The client crashes after sending a request.

- 错误3：服务器宕机
  - 两种情形：执行后宕机、执行前宕机
  - 解决办法：
    - at-least-once semantics：keep try until getting reply
    - at-most-once semantics: give up and report error
    - guarantee nothing: anything may happen
    - exactly-once semantics: ? impossible to guarantee

Exactly-once不可能

没有一种客户策略和服务器策略的结合可以在所有可能的失效事件顺序下正确工作

	Strategy M → P			Strategy P → M		
Reissue strategy	MPC	MC(P)	C(MP)	PMC	PC(M)	C(PM)
Always	DUP	OK	OK	DUP	DUP	OK
Never	OK	ZERO	ZERO	OK	OK	ZERO
Only when ACKed	DUP	OK	ZERO	DUP	OK	ZERO
Only when not ACKed	OK	ZERO	OK	OK	DUP	OK
Client	Server			Server		

OK = Document updated once
DUP = Document updated twice
ZERO = Document not update at all

分布式提交

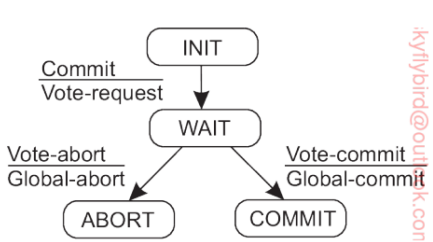
- 问题定义

  一个操作要么被进程组中的每一个成员执行，要么一个都不执行。（原子多播的更一般化抽象）
- 例子：
  - 原子多播： “操作”具体为“消息的交付”
  - 分布式事务： “操作”具体为“事务提交”
- 系统模型：
  - 一个Coordinator，多个Participants
  - 节点/进程会失效
  - 消息不丢失（如果丢失可以通过重发机制处理）

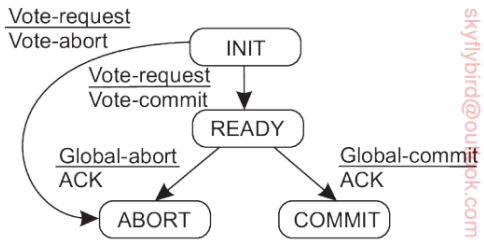
两阶段提交协议

- 失效处理：正常进程被Block后的处理（timeout后预期消息未收到）

角色	Block的状态	Block措施
P	INIT	Send Vote-abort，Abort
C	WAIT	Send Global-abort，Abort
P	READY	a) Wait until C recover, or b) Contact other participant Q to get info.: i) Q COMMIT/ABORT: do the same ii) Q INIT: Abort iii) READY: contact another, if all "REDAY": Cannot decide, wait for C to recover



(a)



(b)

如果所有的参与者都被Block在READY状态，整个协议Block。

三阶段提交协议

- 增加预提交阶段

If all correct participants blocked in “READY”: go to Abort.

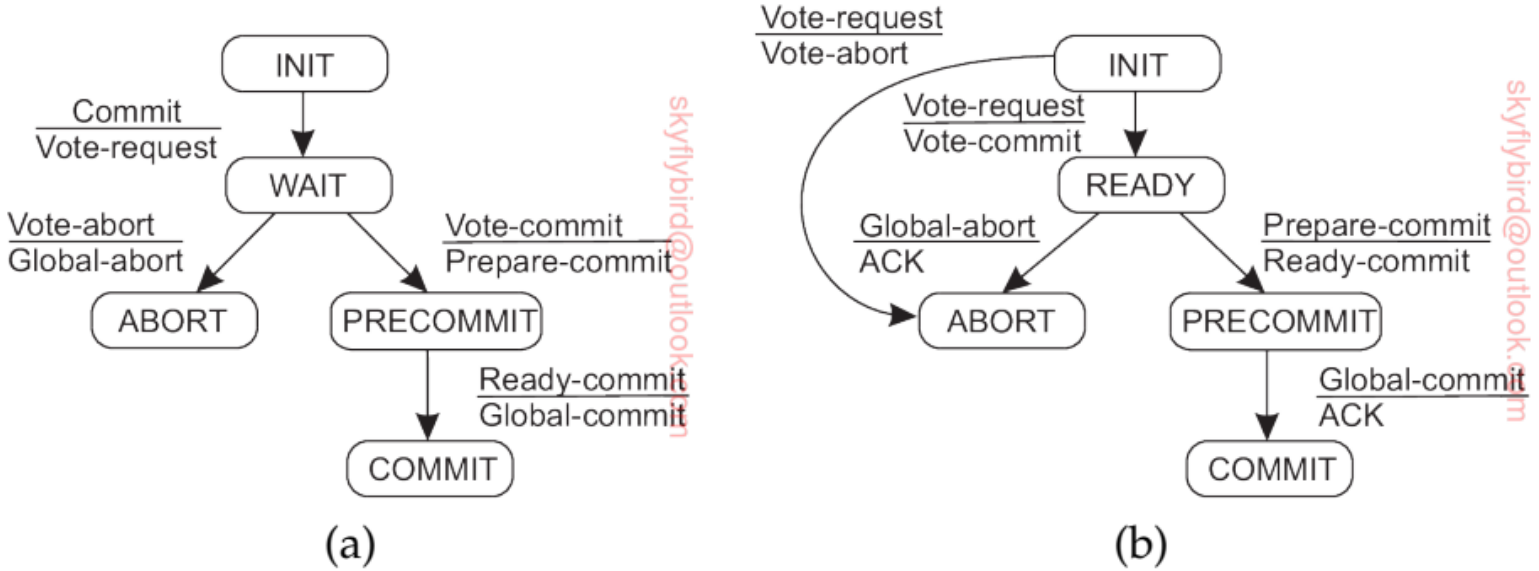


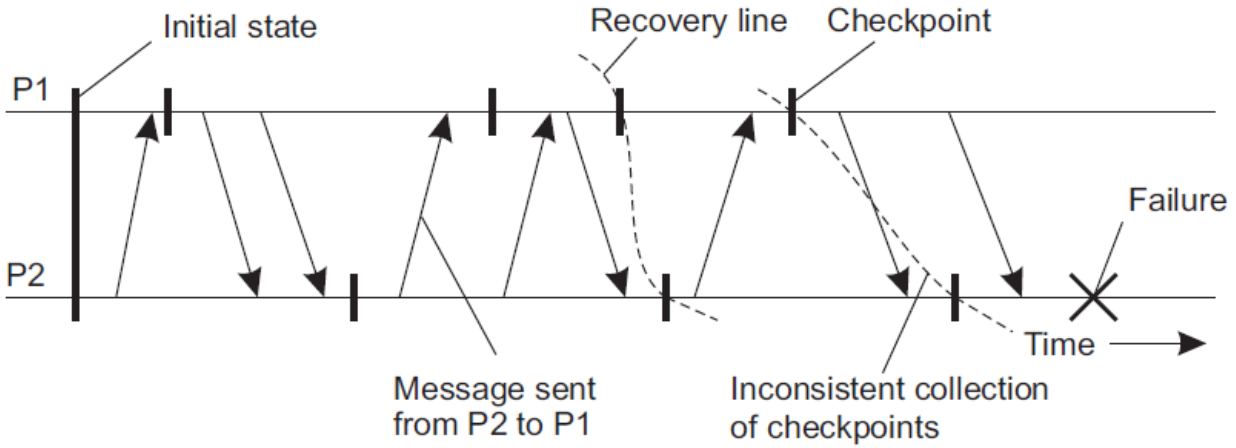
图 8.22 3PC 中协作者和参与者的有限状态机

(a) 3PC 中协作者的有限状态机；(b) 参与者的有限状态机

角色	Block的状态	Block措施
P	READY	Contact other participant Q, if some INIT or all “REDAY”: Abort if all “PRECOMMIT”: Commit

检查点Checkpointing

- 概念
  - 分布式系统的状态记录，即分布式快照（Snapshot），如消息收发。
  - 进程的本地记录构成全局快照。
- 恢复线路（Recovery line）
  - 最近一次的全局一致的检查点。



检查点机制：协调检查点

Essence

Each process takes a checkpoint after a globally coordinated action.

Simple solution

Use a two-phase blocking protocol:

- A coordinator multicasts a checkpoint request message
- When a participant receives such a message, it takes a checkpoint, stops sending (application) messages, and reports back that it has taken a checkpoint
- When all checkpoints have been confirmed at the coordinator, the latter broadcasts a checkpoint done message to allow all processes to continue

Observation

It is possible to consider only those processes that depend on the recovery of the coordinator, and ignore the rest

独立的检查点

Essence

Each process independently takes checkpoints, with the risk of a cascaded rollback to system startup.

- Let $CP_i(m)$ denote m^{th} checkpoint of process P_i and $INT_i(m)$ the interval between $CP_i(m-1)$ and $CP_i(m)$.
- When process P_i sends a message in interval $INT_i(m)$, it piggybacks (i, m)
- When process P_j receives a message in interval $INT_j(n)$, it records the dependency $INT_i(m) \rightarrow INT_j(n)$.
- The dependency $INT_i(m) \rightarrow INT_j(n)$ is saved to storage when taking checkpoint $CP_j(n)$.

Observation

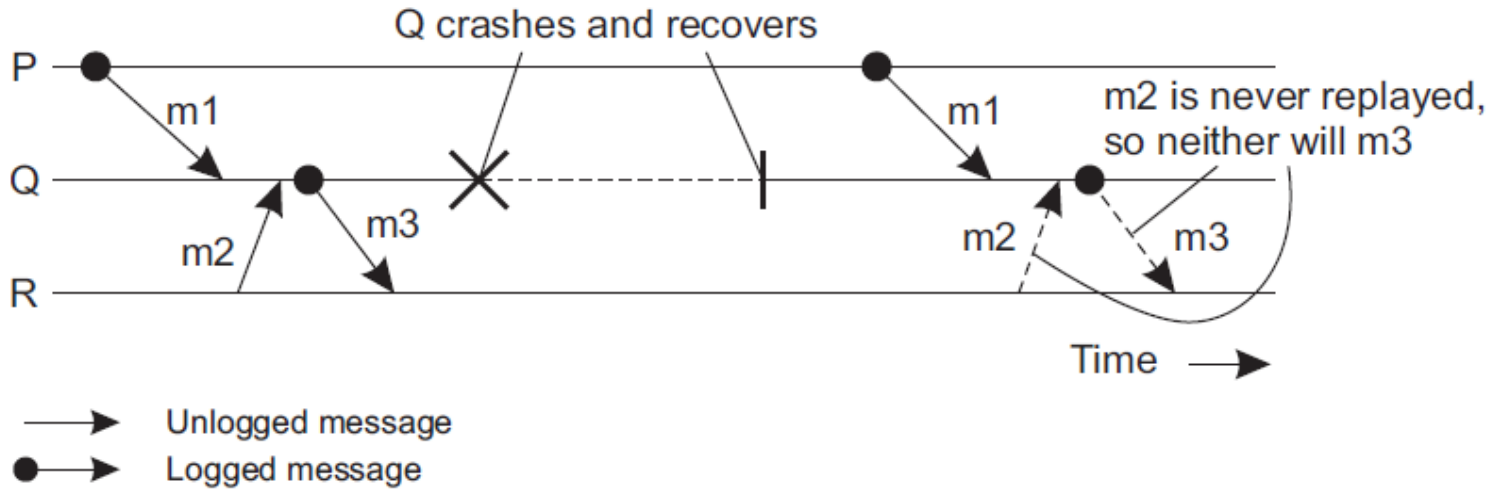
If process P_i rolls back to $CP_i(m-1)$, P_j must roll back to $CP_j(n-1)$.

消息日志

When should we actually log messages?

Avoid orphan processes:

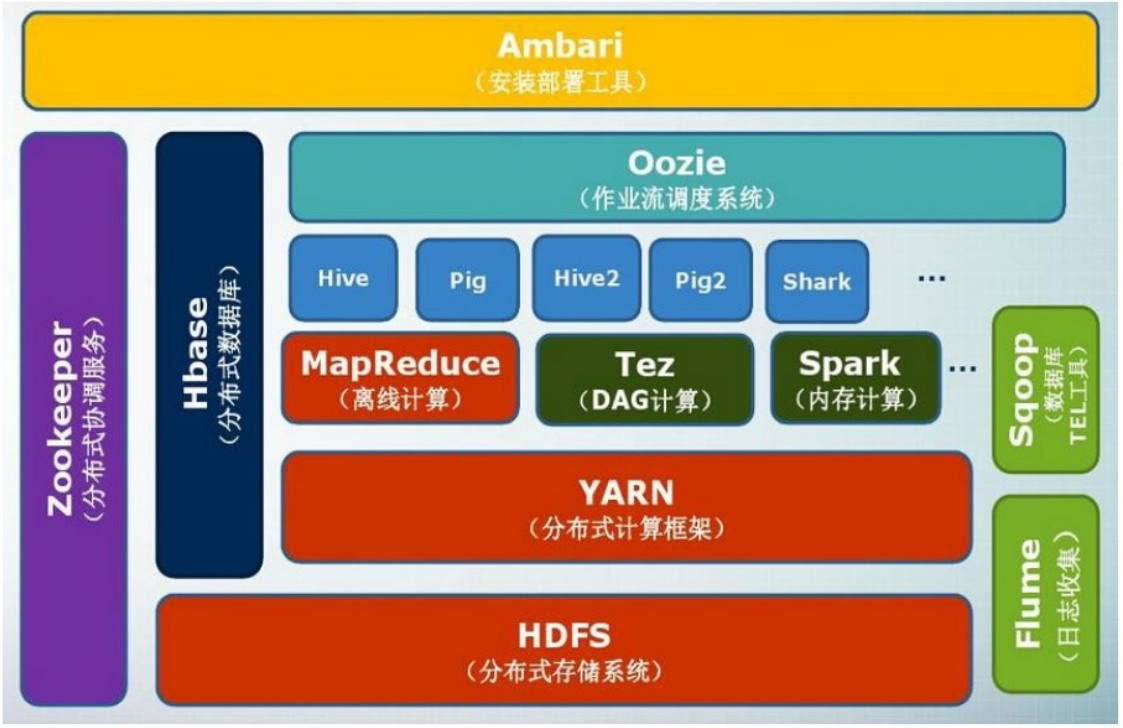
- Process Q has just received and delivered messages m_1 and m_2
- Assume that m_2 is never logged.
- After delivering m_1 and m_2, Q sends message m_3 to process R
- Process R receives and subsequently delivers m_3: it is an orphan.

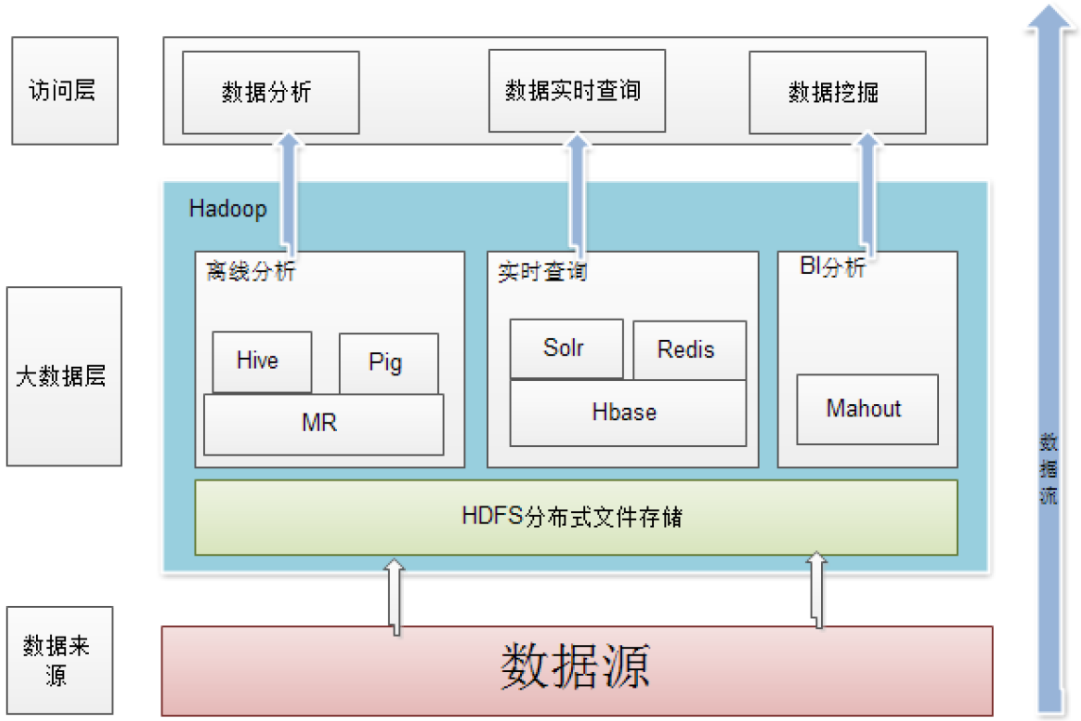


Part III 系统实例

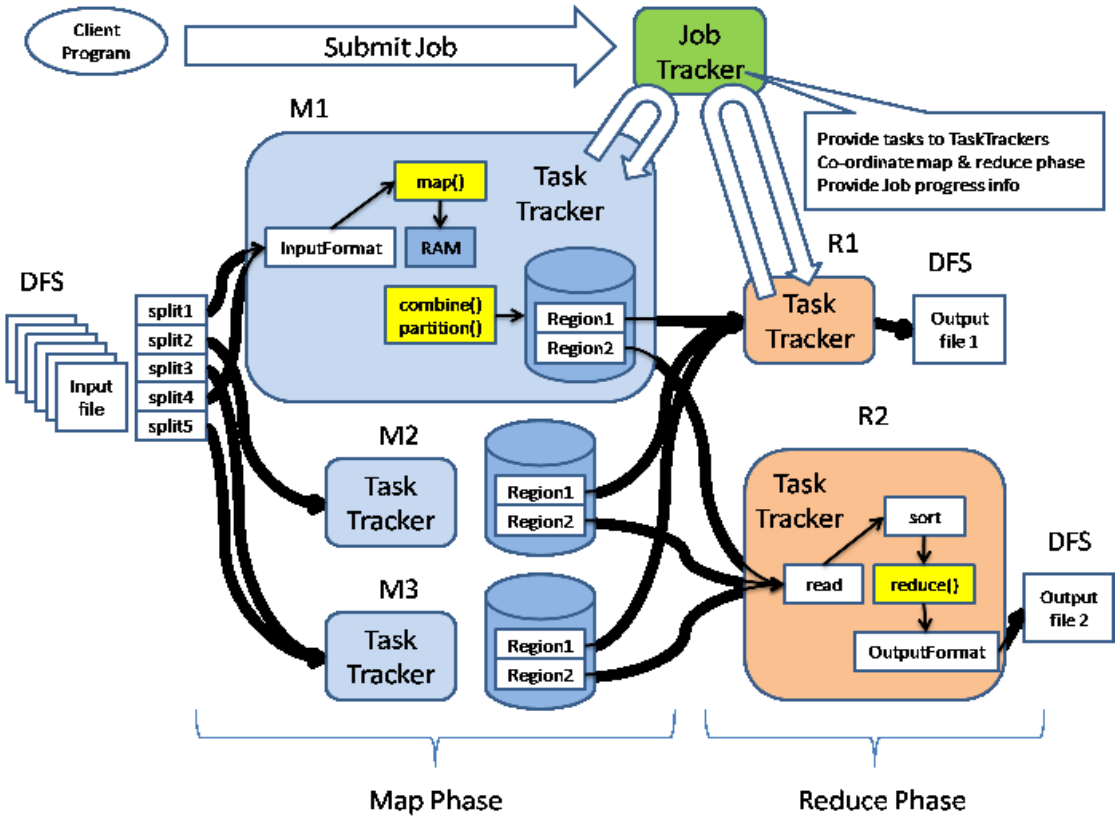
- 大数据处理系统
- 区块链系统

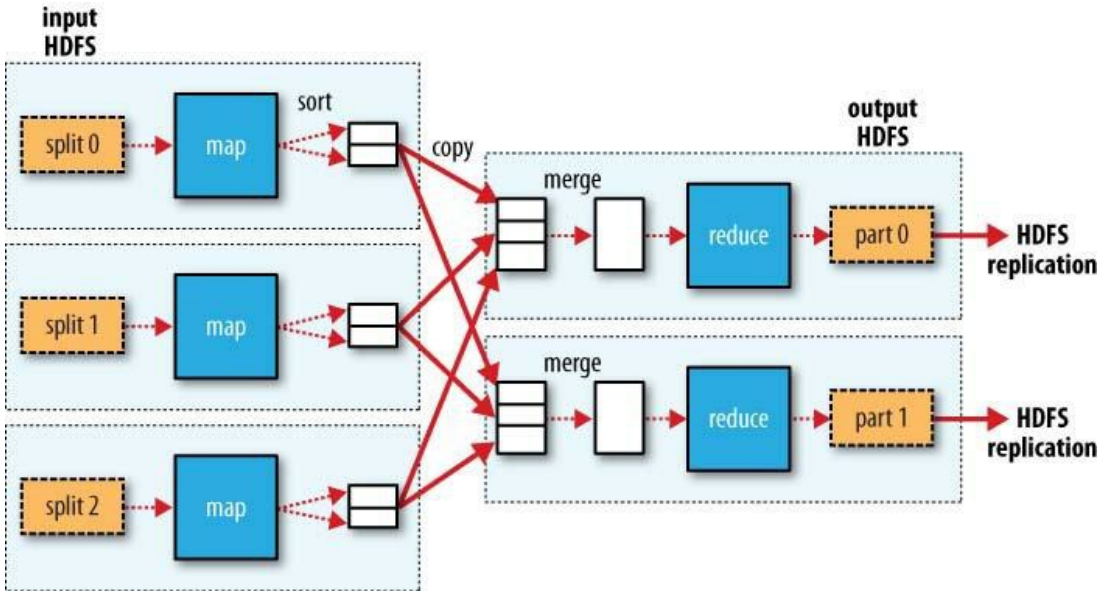
Hadoop Ecosystem





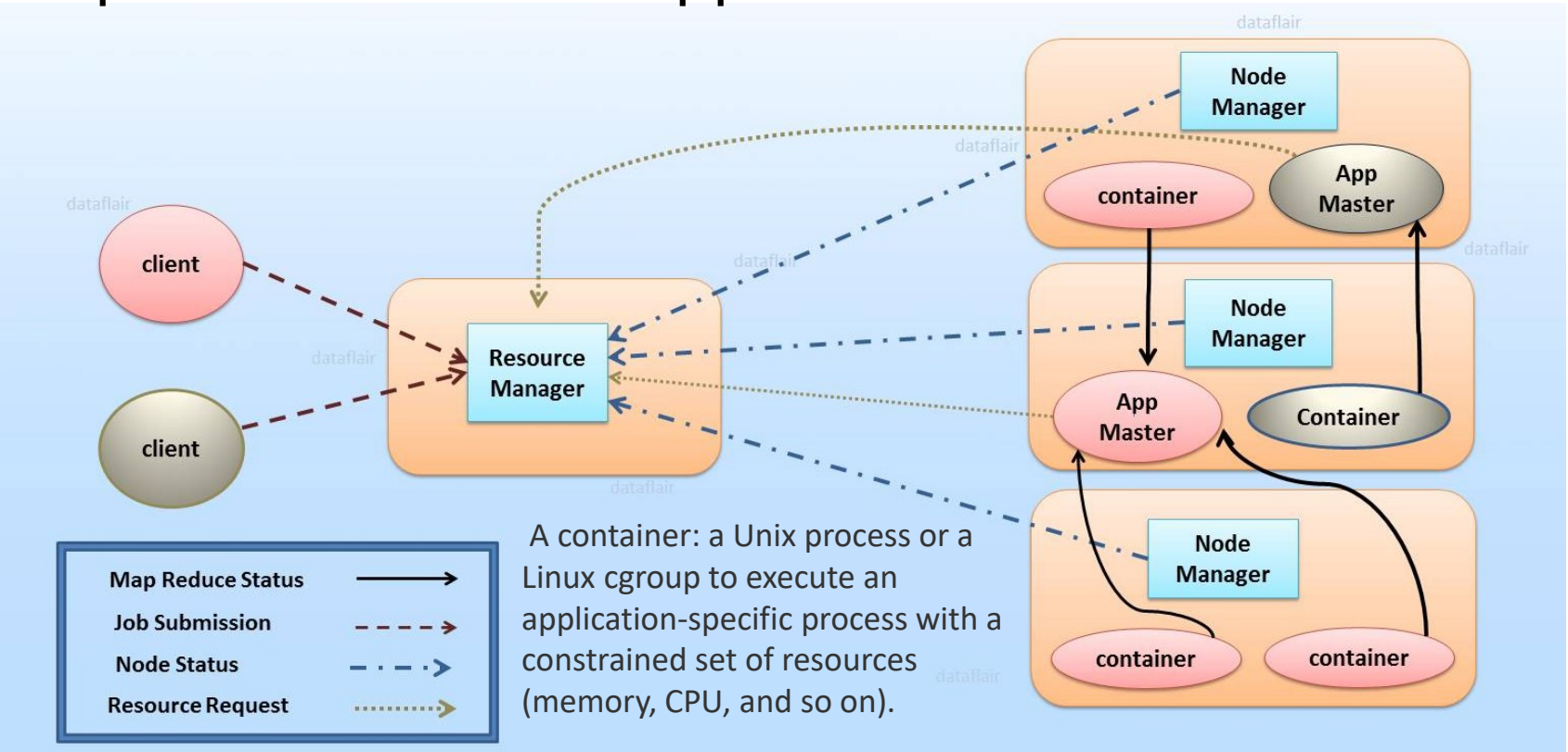
MapReduce Framework



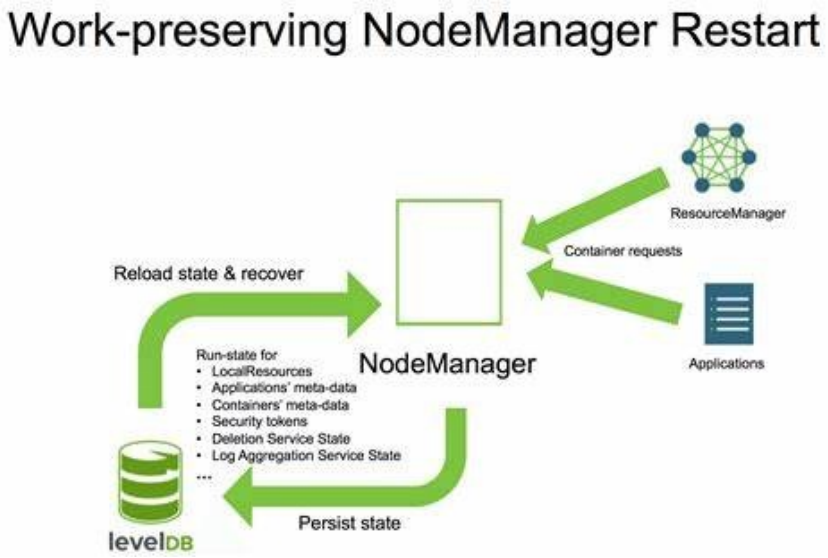


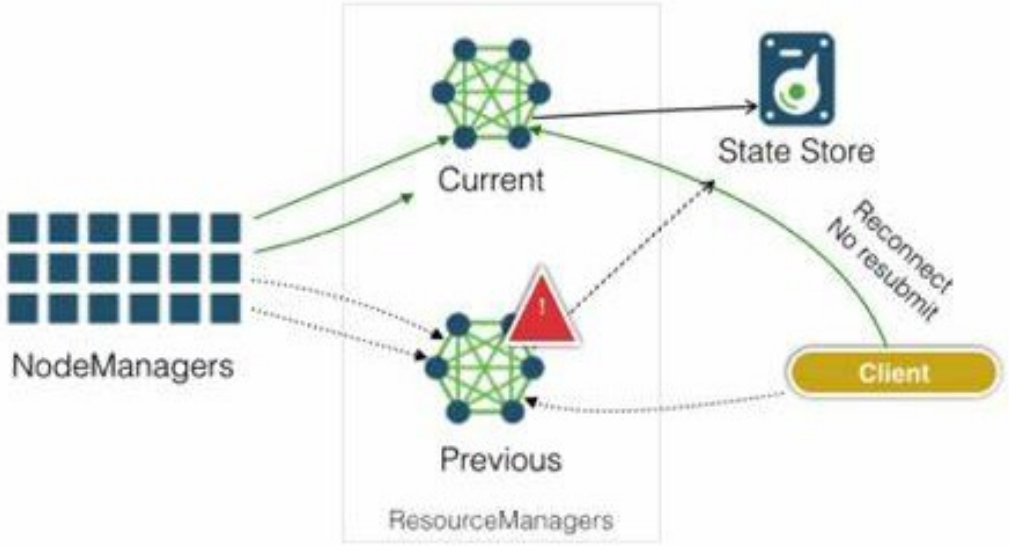
YARN Architecture

- Resource Manager: a master daemon
- Node Manager: slave daemon, one per salve node
- Application Master: one per application
- Container: process to execute app



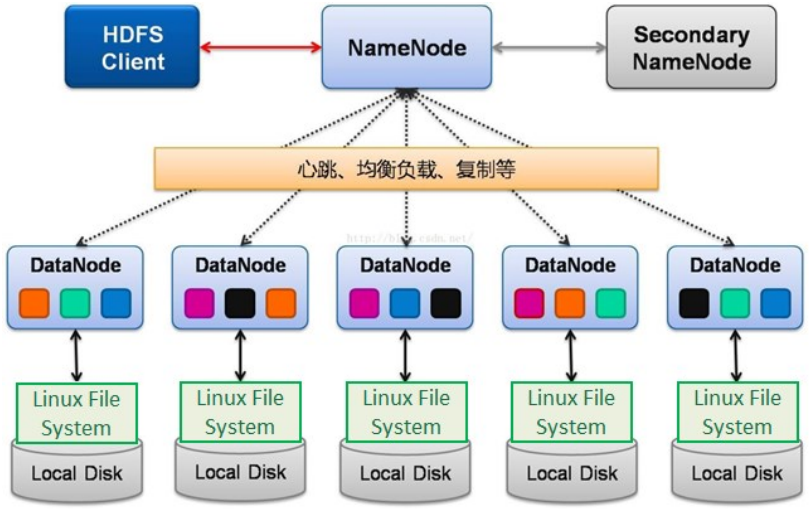
Yarn容错机制

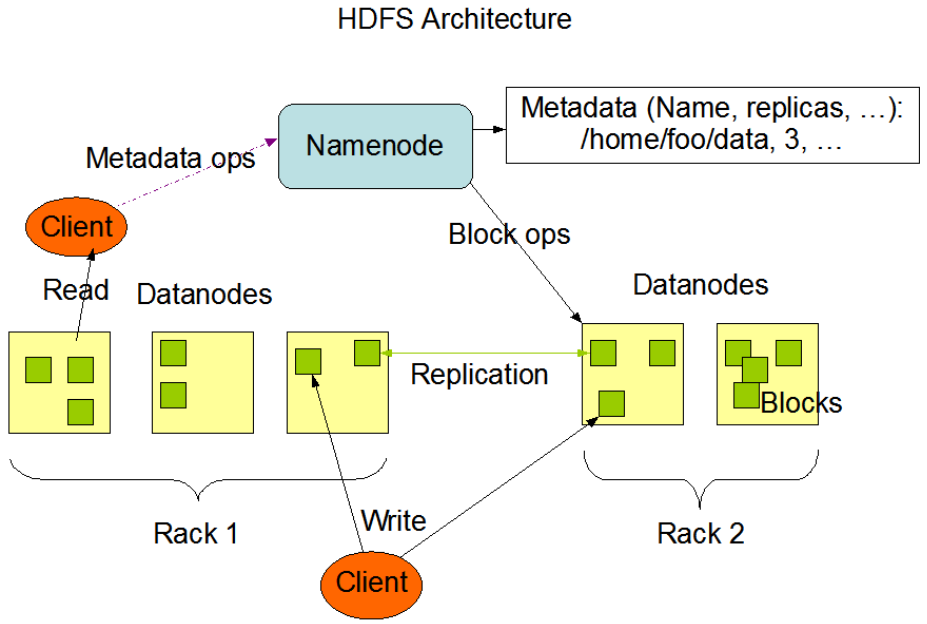

Work-preserving NodeManager Restart
ResourceManager
Container requests
Reload state & recover
NodeManager
Applications
Run-state for
• LocalResources
• Applications' meta-data
• Containers' meta-data
• Security tokens
• Deletion Service State
• Log Aggregation Service State
...
Persist state
levelDB


Current
State Store
Reconnect
No resubmit
NodeManagers
Client
Previous
ResourceManagers

HDFS系统

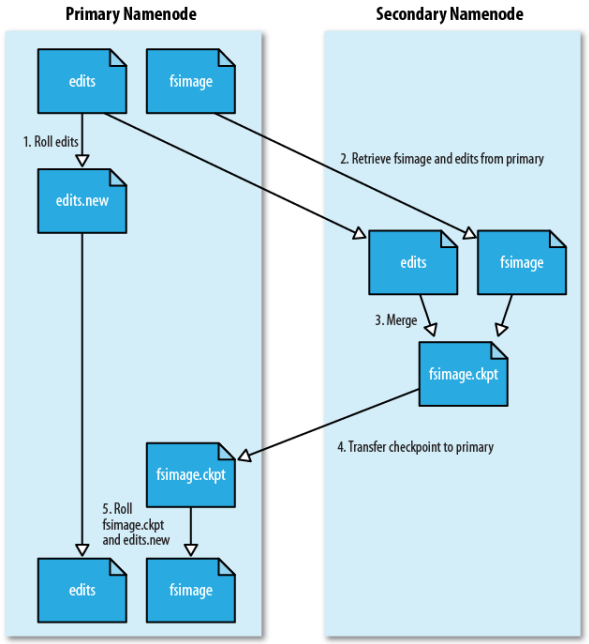
- Master-Slave架构
- 节点的本地OS具体管理本地资源
- HDFS处于中间件层次，整合集群存储

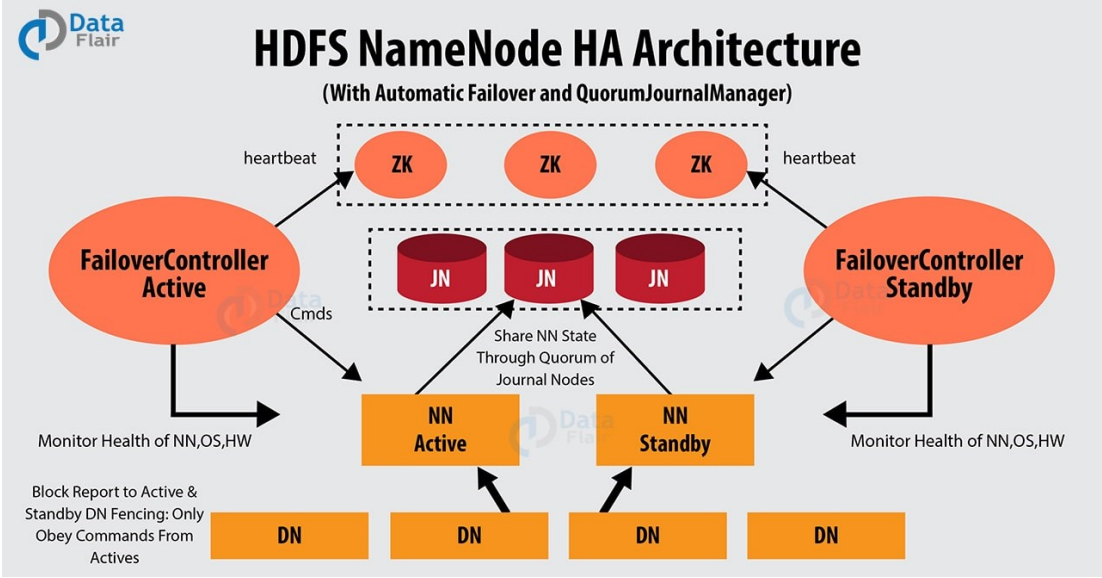




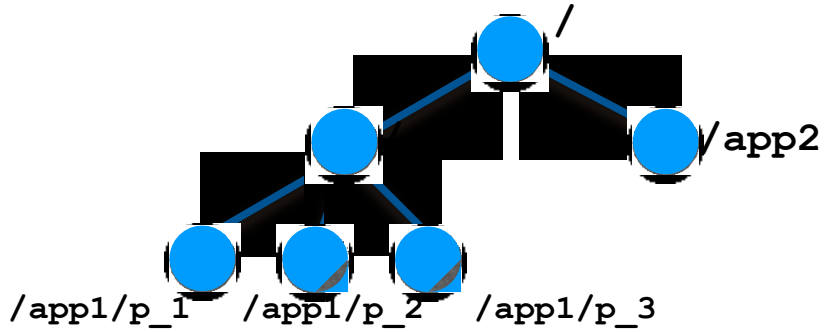
HDFS FT and HA

- DataNode Failure:
  - Replication, heartbeat
- Data Integrity:
  - Checksum
- Metadata Failure:
  - FsImage and EditLog, Checkpoint
- NameNode HA
  - Zookeeper+ Journal node set

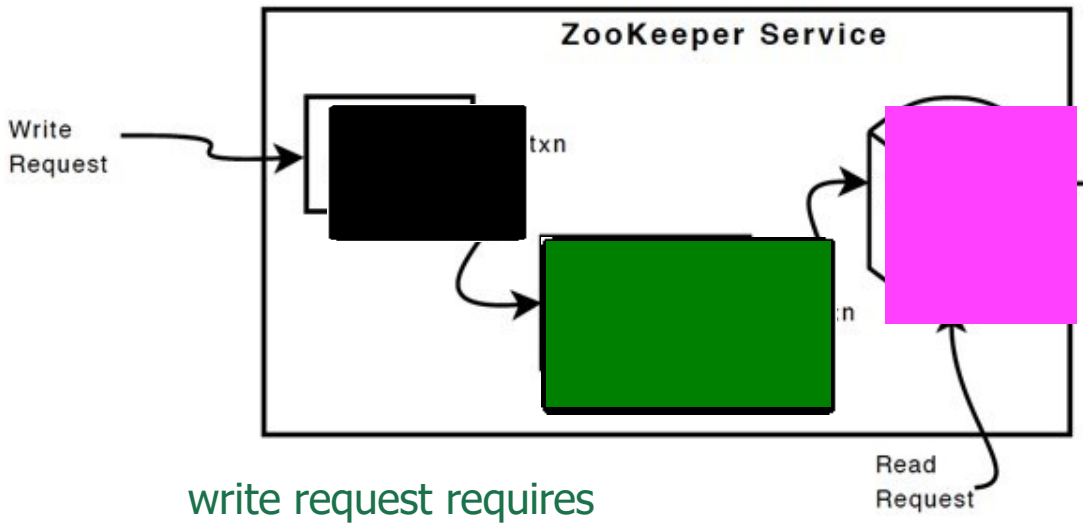




ZooKeeper



All znodes store **data (file like)** & can have **children (directory like)**.



replicated across all servers (in-memory)

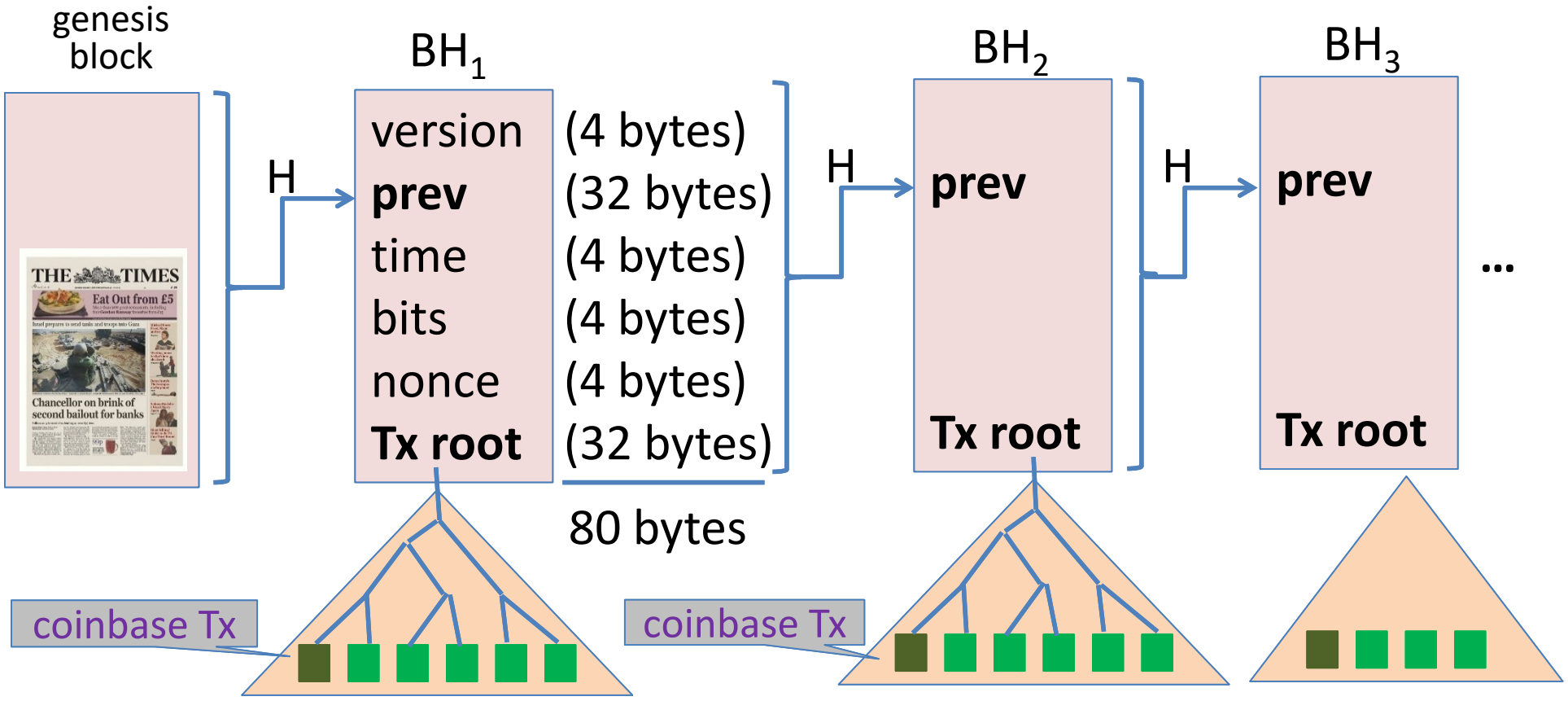
updates first logged to disk (write-ahead log), and snapshot for recovery

write request requires coordination among servers

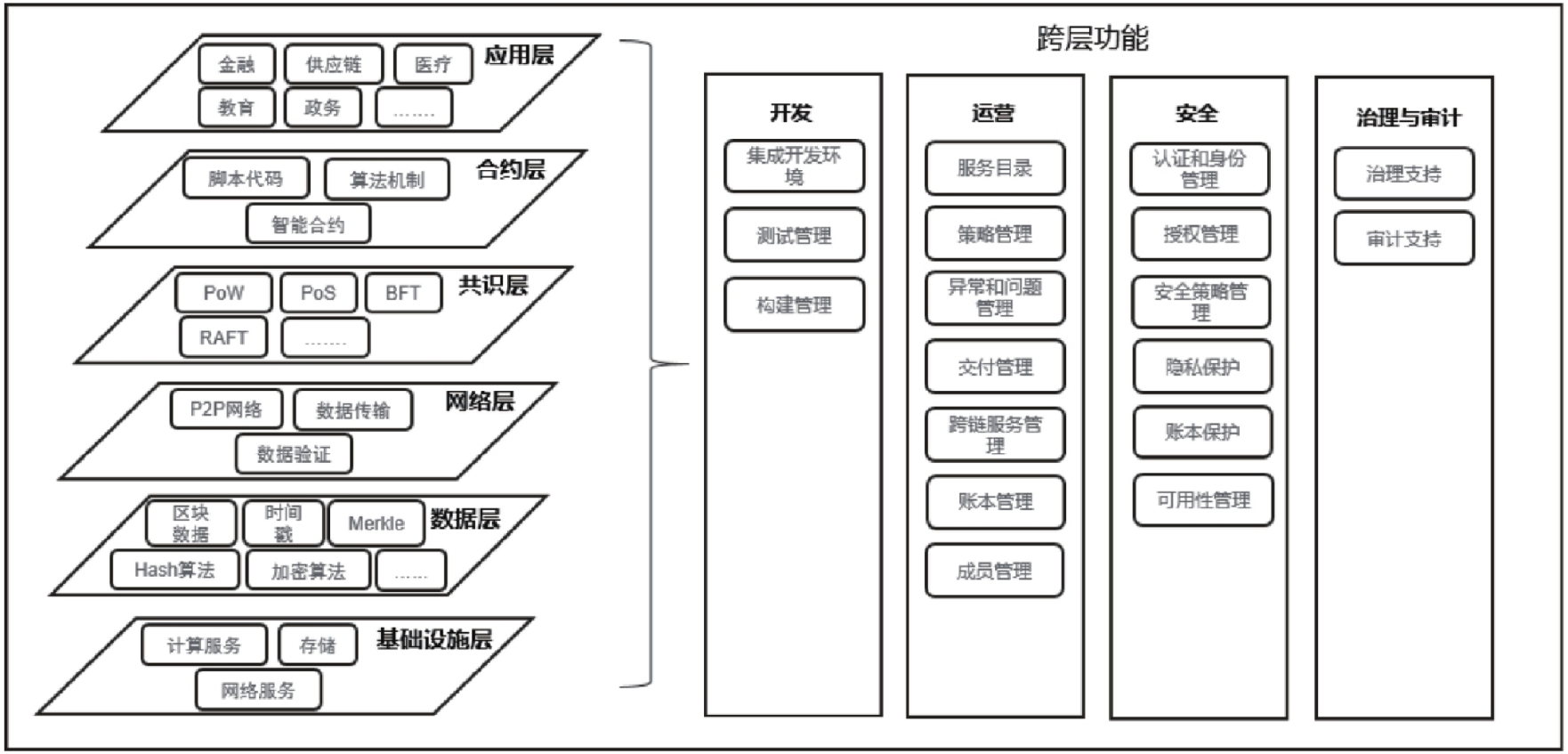
区块链

Bitcoin blockchain:

- a sequence of block headers, 80 bytes each
- connected via Hash values, new block every ≈10 minutes



软件架构



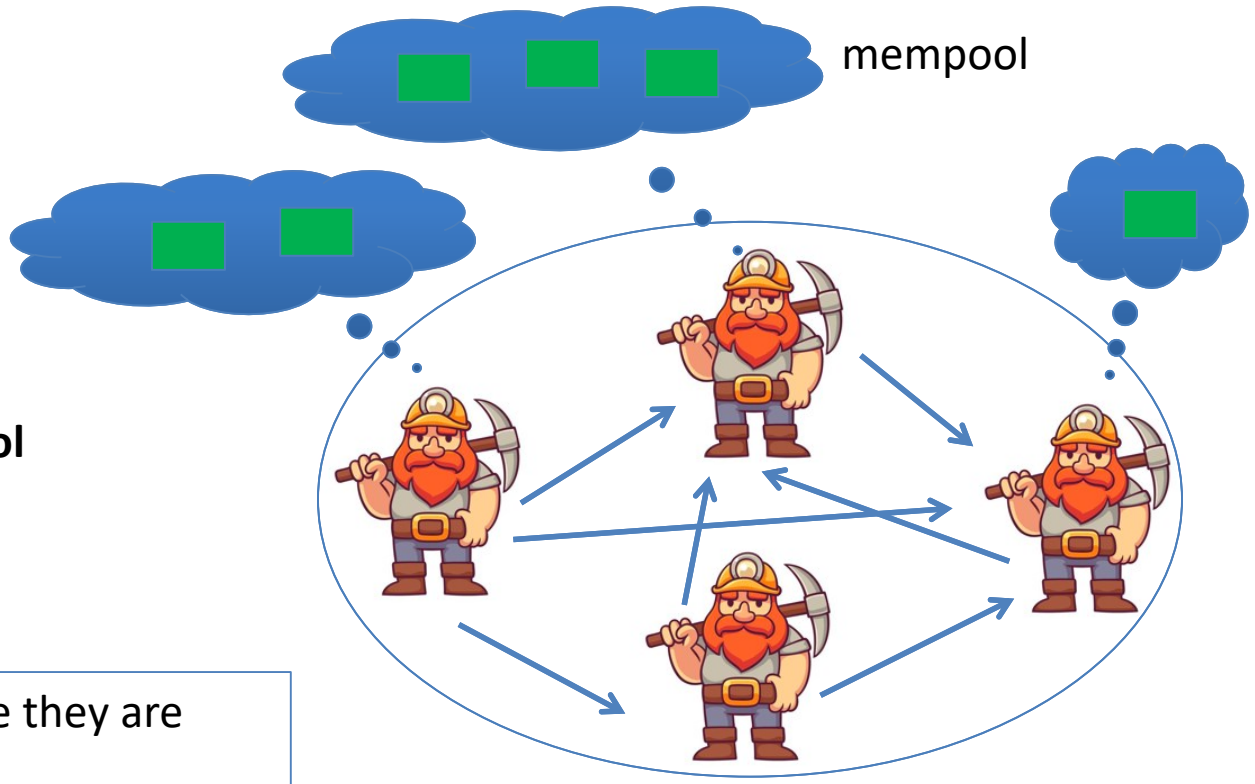
区块链的六层架构

Bitcoin consensus layer

Miners broadcast received Tx to the P2P network

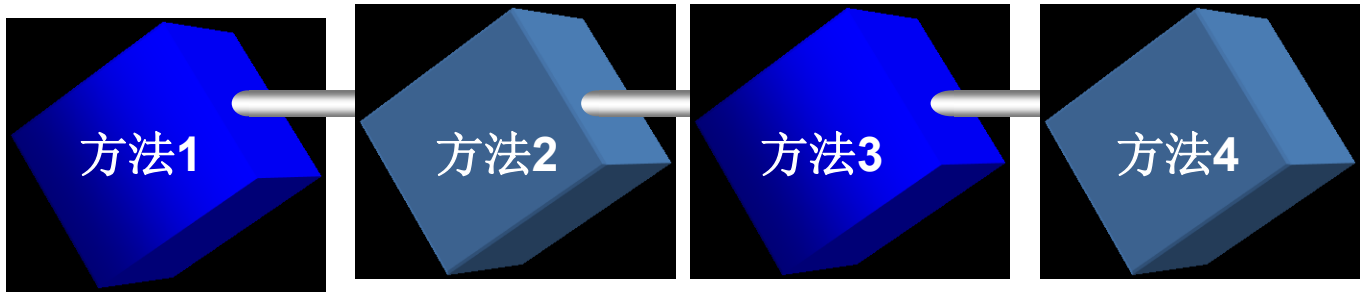
Every miner:
- validates received Tx and stores them in its **mempool** (unconfirmed Tx)

note: miners see all Tx before they are posted on chain



Bitcoin P2P network

3. 高扩展技术



区快分片（Sharding）	链下计算（Off-chain computing）及侧链（Sidechain）	扩展性共识（Scalable consensus）	并发链结构 有向无环结构（DAG）

结束!

预祝取得好成绩!